



UiA University
of Agder

FortGjort

Group 0022

written by

Nancy Rønqist Erichsen

Adam Hazel

Gunn Marita Jomås-Britten

Nora Lior

in

IKT205-G 25V

Application Development

With guidance from Yuan Lin

Faculty of Engineering and Science

University of Agder

Grimstad, May 2025

Abstract

This project addresses the everyday challenge many families face: managing household chores in a structured and motivating way. Despite the existence of several chore-related apps, many suffer from high complexity, subscription fees, or poor support for families with younger children. As a result, there is a need for a free, easy-to-use solution that enables parents to assign tasks and rewards while keeping children engaged through a simple and playful interface.

To meet this need, a mobile application, FortGjort, was developed using React Native and Firebase. The app introduces a system where parents can create and manage child profiles, assign tasks with deadlines and point values, and define custom rewards. Children interact with the system through a separate, simplified view tailored to their age group, allowing them to track tasks and progress in an intuitive way. PIN-based access control ensures that parental functions are protected from unintended use.

The core functionalities implemented include user management, task creation and tracking reward systems. The solution prioritizes accessibility, data security, and user experience, making it practical for everyday use by real families. The development process followed an iterative methodology, balancing feature development with continuous testing and refinement.

The result is a working MVP that lays the groundwork for future improvements such as notifications, media upload for task verification, and shared household access. The app has been structured for scalability and is now being prepared for release through Google Play as part of the project completion.

In conclusion, this application offers a meaningful tool to support collaboration between parents and children in everyday routines. Its design focuses on usability, adaptability, and real-world functionality, making it a valuable starting point for further development beyond the scope of the initial project.

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja
2.	Vi erklærer videre at denne besvarelsen: • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høgskoler i Norge, jf. Universitets- og høgskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høgskolens retningslinjer for behandling av saker om fusk.	Ja
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).

Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Ja
Er oppgaven båndlagt (konfidensiell)?	Nei
Er oppgaven unntatt offentlighet?	Nei

Preface

This project, FortGjort, was developed as part of the fourth semester in the computer engineering course IKT205: Application Development at the University of Agder.

The goal was to design, with the help of various technologies and frameworks, a simple-to-use mobile application for families to administrate household chores in a way that boosts external motivation for the involved children.

The project was carried out by Nancy Rønqist Erichsen, Adam Hazel, Gunn Marita Jomås-Britten, and Nora Lior, with work conducted both at UiA Campus Grimstad and remotely. Tools such as Git, Taiga, and Figma were used throughout the project.

We extend our gratitude to Yuan Lin for facilitating this challenge, which provided us with valuable learning opportunities.

Grimstad, September 7, 2026

Nancy Rønqist Erichsen, Adam Hazel, Gunn Marita Jomås-Britten, and Nora Lior.

Contents

1	Introduction	1
2	Limitations	2
3	Theoretical Background	3
3.1	Private Households	3
3.2	Chores	3
4	Method	5
4.1	Process	5
4.1.1	Waterfall	5
4.1.2	Scrum as a Sub-Process	5
4.2	Tools Used	5
4.2.1	Project Tracking	5
4.2.2	Teamwork	5
4.2.3	Product and Report Creation	6
4.3	Project Timeline	6
4.3.1	Idea Phase	6
4.3.2	Planning	6
4.3.3	Core and Advanced Development	7
4.3.4	Final Adjustments	7
4.3.5	Testing and Debugging	7
4.3.6	Reporting	7
4.4	Roles and Responsibilities	7
4.5	Research	8
4.5.1	Technology Stack	8
4.5.2	Competitive Analysis	8
4.5.3	UI/UX Design	8
5	Research Results	9
5.1	Technology Stack: Frontend	9
5.2	Technology Stack: Backend	10
5.3	Technology Stack: Local Database	12
5.4	Technology Stack: Remote Database and Hosting	12
5.5	Competitive Analysis Results	13
5.6	UI/UX Research Results	14
5.6.1	User Experience and User Interface	14
5.6.2	Designing for Children	15

6	Requirements	16
6.1	Functional Requirements	16
6.1.1	App Access	16
6.1.2	Task Management	17
6.1.3	Reward System	17
6.2	Non-Functional Requirements	18
7	Solution	19
7.1	Entity Model	19
7.2	Architecture	20
7.3	App Structure and Routing	21
7.4	Product	23
7.4.1	App Access	23
7.4.2	Tasks Management	25
7.4.3	Reward System	30
8	Technical Background	33
8.1	Choice of Technologies	33
8.2	React Native	33
8.3	Firebase	33
8.4	Expo	34
8.5	Maestro	34
8.6	Routing with Expo Router	35
8.7	Expo Crypto	35
9	Security	37
10	Implementation	38
10.1	Entity Model	38
10.2	Architecture	39
10.3	App Structure and Routing	40
10.4	App Access	43
10.5	Task Management	50
10.6	Reward system	57
11	Testing and validation	64
12	Discussion	66
12.1	Process	66
12.2	Product	66
12.3	Implementation	67

12.4 Challenges	68
12.5 For the future	69
13 Conclusion	70
Appendix A Product Vision	75
Appendix B Group Contract	75
Appendix C Product Info	75
Appendix D Meeting Minutes	75
Appendix E Demo Video	75
Appendix F Git Shortlog	75
Appendix G Scrum List	75
Appendix H Requirements	75
Appendix I Figma Views	76
Appendix J Security Analysis	76
Appendix K Add Reward Sequence Diagram	76
Appendix L Test Report	76
Appendix M Entity Model	76
Appendix N Individual Reports	76
Appendix O Time Sheets	76
Appendix P Project Brainstorming	77
Appendix Q Use Case Descriptions	77

List of Figures

1	Distribution of household types in EU countries, 2007 [25]	3
2	Waterfall process stages	5
3	Project timeline overview	6
4	User story example with effort and priority scores	7
5	Overview of frontend frameworks evaluated	9
6	An overview of backend technologies	11
7	Comparison of traditional backend and Firebase [52]	11
8	Entity model for FortGjort	20
9	Component based architecture[29]	21
10	Conceptual views of parent navigation design	22
11	Conceptual views of child navigation design	23
12	View of solution: User registration	24
13	View of solution: Log in	24
14	View of solution: Create household	24
15	View of solution: Create new household	24
16	View of solution: Household home screen	25
17	View of solution: Part of the process of a parent creating a new task, adding task name and points	26
18	View of solution: Part of the process of a parent creating a new task, assigning it to one or more child(ren)	26
19	View of solution: Part of the process of a parent creating a new task, choosing if it should be visible to the child(ren) or not	26
20	View of solution: Part of the process of a parent creating a new one- time task, selecting date for completion	27
21	View of solution: Part of the process of a parent creating a new recurring task, selecting weekdays the task should be completed	27
22	View of solution: A parent can view the task that a child has completed	28
23	View of solution: A parent can approve a task that a child has completed	28
24	View of solution: A parent can delete a task that is not completed by a child	29
25	View of solution: A parent can delete a task that is completed and approved	29
26	View of solution: A child may view their assigned tasks by clicking on "oppgaver" in the menu after entering their child profile.	30
27	View of solution: A child can mark the task as completed or uncom- pleted	30
28	View of solution: List of available rewards	31
29	View of solution: A parent can add a new reward to the reward-bank	31
30	View of solution: List of available rewards	32

31	View of solution: Child buy reward	32
32	View of solution: Purchased but not received rewards	32
33	View of solution: Rewards waiting for approval	32
34	View of solution: A parent can mark a reward as used	32
35	Overview - Security analysis	37
36	Simplified visualization of file-based routing in app/(parent)/(tabs)/profile	41
37	The tab bar in parent view	41
38	Child homescreen with the menu button in left corner	43
39	The menu popping up, when clicking the star button	43
40	Registration and household creation flow in FortGjort	44
41	Email confirmation screen with resend option	45
42	Sequence diagram illustrating the flow of a parent logging into the parent view.	48
43	Set new PIN	49
44	Screen for when parent has a PIN	49
45	Enter PIN, to enter parent view	49
46	Relationship between tasks and household	51
47	Listing tasks for parents	54
48	Listing tasks for children	54
49	State diagram for a task	56
50	Trying to delete a completed, but pending task	57
51	Firestore structure for rewards per household	58
52	Sequence diagram showing the process of adding a reward	58
53	Flow chart of a child performing a reward purchase	60
54	UI showing purchased rewards ("backpack")	61
55	Toggle reward to used	63
56	A reward has been toggled to used, and the status of the top reward, "Sjokolade", has updated in the UI behind the modal	63
57	Test cases 1-11	64
58	Test cases 12-28	65

List of Tables

1	App Access Requirements	16
2	Task Management Requirements	17
3	Reward System Requirements	17
4	Non-Functional Requirements	18

Code Snippets

1	The Child model	39
2	Retrieving a single child document by its ID	39
3	A shortened example of how the floating menu logic was implemented using React Native	42
4	Creating a Firebase Auth user and Firestore Parent document	44
5	Polling logic in useEmailVerification()	45
6	Login logic with routing based on verification and household mem- bership	46
7	Depending on the presence of a pinHash , the interface dynamically displays the option to either create, update, or remove the PIN. The PIN get securely hashed and stored	47
8	The application performs a check to determine whether the parent has configured a PIN.	47
9	Session detection in WelcomeScreen.tsx	50
10	Loading state during authentication check	50
11	Mapping weekday codes to JavaScript's numeric day values	51
12	Returns dates within a range that match the selected weekdays . . .	52
13	Creating a task	52
14	Filtering tasks in child view	53
15	Filtering tasks in parent view	53
16	An excerpt of what happens when a parent approves a task	55
17	Checking if a task can get deleted	57

1 Introduction

FortGjort, defined in the context of this report as age-appropriate tasks for the home environment, have been a part of daily family life for generations. However, surveys suggest a decline in children's participation in chores [59].

While the reasons for this decline vary from family to family, this report presents a mobile application that aims to help families establish a clear structure for assigning and managing chores in their household, whilst providing external motivation for children through a reward system.

The following questions, among others, will be addressed:

- What are the characteristics of the minimum viable product?
- What methods and processes were used throughout this project?
- What are the weaknesses of the implemented solution, and how can these weaknesses be mitigated in future iterations of the product?
- What was learned throughout the completion of this project?

This project was completed as an exam project for the subject "Application Development (IKT205)" at UiA Grimstad.

Product Vision

FortGjort is an interactive application designed to motivate children aged 6–15 to participate in household chores while providing parents with an effective tool to manage and oversee these activities. The app combines engaging, child-friendly elements with robust parental controls, ensuring an accessible and interactive experience for the entire family.

Key features include a user-friendly interface and flexible parental controls, such as customizable task management, point assignment, deadlines, and recurring chores. In the application, a single registered user (with email and password) should be able to create and manage multiple child profiles. It should also be possible for multiple guardians to access the same child profile. Additionally, cloud-based synchronization ensures that data is accessible across multiple devices while emphasizing secure data storage and compliance with privacy regulations. Unlike many competitors, the app is free to use, with no hidden costs or subscription fees.

A full product vision can be found in Appendix A, as well as directly here: *Product Vision*.

2 Limitations

This project faced certain limitations that affected both the development process and the final outcome. The most significant limitations were related to time constraints and technical expertise with frameworks and libraries.

- **Time Constraints:** The development was conducted within a limited time-frame, which placed restrictions on the breadth of features and the depth of testing that could be completed. As a result, certain functionalities were either simplified or left out to meet the deadline.
- **Technical Experience:** The project team had limited prior experience with React Native, Firebase, and various npm packages used to implement specific features. As a result, significant time was spent learning these tools during the development phase, which affected efficiency and led to challenges in areas such as automatic testing, component integration, and performance optimization.
- **Testing and Scalability:** Due to resource constraints, extensive testing under real-world conditions, including performance testing with higher user loads, could not be carried out. This limited the team's ability to validate the system's scalability and stability across a range of devices and usage scenarios.

3 Theoretical Background

3.1 Private Households

Eurostat [64, 25] classifies households into two main groups – with or without children under 18 – resulting in seven subcategories. Figure 1 shows the distribution across EU countries in 2007:

	Household composition: percentage of households							Household size	
	No children under 18 in household					Children under 18 present		Mean over individuals (8)	Mean over households (9)
	Single adult under 65 (1)	Single adult aged 65+ (2)	Couple both under 65 (3)	Couple at least one 65+ (4)	Other, no under-18s (5)	Single adult with children (6)	2+ adults with children (7)		
Sweden	24.0	15.6	16.6	11.8	5.7	4.2	22.0	2.8	2.1
Finland	25.6	13.0	19.7	10.1	7.6	3.4	20.8	2.9	2.1
Denmark	30.2	14.0	16.5	9.9	4.4	4.8	20.2	2.7	2.0
Netherlands	23.5	11.7	17.0	11.1	10.0	2.8	23.9	3.0	2.3
UK	16.7	13.6	16.6	10.3	12.8	5.4	24.7	3.1	2.4
France	20.0	14.2	15.9	11.2	11.0	3.5	24.2	3.0	2.3
Germany	24.4	14.0	14.7	14.2	11.5	3.1	18.1	2.7	2.1
Austria	21.7	13.4	12.5	10.2	15.8	3.5	23.0	3.1	2.3
Belgium	20.6	13.5	15.6	10.4	13.4	3.8	22.7	3.1	2.3
Luxembourg	18.0	10.9	13.7	10.4	14.9	2.4	29.7	3.1	2.5
Ireland	11.3	10.1	9.5	7.3	20.5	7.1	34.4	3.6	2.8
Italy	14.1	15.0	8.5	11.1	24.2	1.9	25.1	3.1	2.4
Spain	8.6	8.7	12.2	10.0	29.2	1.1	30.2	3.3	2.8
Portugal	6.4	10.6	9.5	12.1	26.5	2.0	33.0	3.3	2.8
Greece	10.4	9.7	8.8	12.3	29.9	1.0	28.0	3.3	2.7
Cyprus	8.9	7.2	9.6	11.9	25.3	1.9	35.4	3.6	2.9
Czech Republic	12.4	11.4	14.4	10.0	22.2	2.9	26.7	3.1	2.5
Hungary	11.5	12.8	12.8	8.6	22.6	3.2	28.6	3.3	2.6
Estonia	18.3	15.4	11.1	7.8	19.1	4.2	24.2	3.1	2.3
Latvia	12.8	12.4	8.6	6.5	25.7	4.0	30.1	3.4	2.6
Lithuania	12.1	14.9	9.6	7.9	21.9	3.8	29.8	3.3	2.6
Slovenia	9.0	11.8	7.8	8.8	30.8	2.0	29.9	3.5	2.8
Slovakia	11.4	13.1	8.0	7.9	30.1	1.3	28.2	3.7	2.8
Poland	11.3	13.4	10.0	6.6	24.6	1.8	32.4	3.8	2.8

Figure 1: Distribution of household types in EU countries, 2007 [25]

Although slightly outdated, the data remains relevant. In Norway, 24% of households had children under 18 as of September 2024 [17], supporting the relevance of the app’s target audience.

3.2 Chores

Participation and Benefits

Research on chore participation is limited and inconsistent. Some surveys suggest that many parents do not require their children to do chores despite having done them themselves [60, 62].

Nevertheless, studies generally highlight the positive effects of chore involvement, including greater self-competence, prosocial behavior, and executive function development [63, 54].

Motivation and External Rewards

While the app uses external motivation like points and rewards, it is important to acknowledge that such incentives may reduce intrinsic motivation and the social value of chores [42]. Still, many families use rewards as part of their routines, and the app aims to support this in a structured and engaging way.

4 Method

4.1 Process

4.1.1 Waterfall

The group chose a modified waterfall model for the project, following its linear stages, see Figure 2, but allowing for occasional revisits to earlier phases. For a practical timeline, see Section ??.

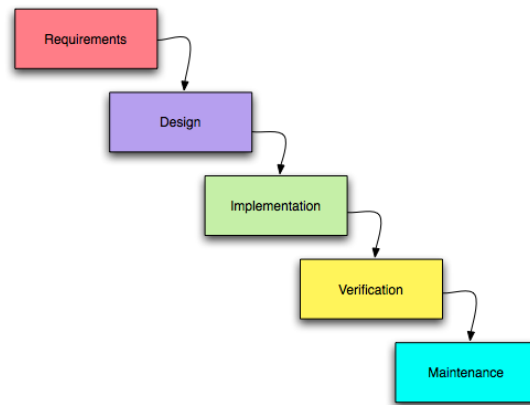


Figure 2: Waterfall process stages

4.1.2 Scrum as a Sub-Process

During implementation, Scrum was used as a sub-process. After initial planning that included user stories, UI sketches, work continued iteratively. This enabled early development start, continuous adjustments, and combining implementation with verification for faster bug handling.

4.2 Tools Used

4.2.1 Project Tracking

- **Microsoft Excel:** Used for report progress and scheduling.
- **Taiga:** Scrum board for backlog management and sprint planning.

4.2.2 Teamwork

The following tools were used to facilitate teamwork:

- **Microsoft Teams:** Used for remote meetings.

- **Git with GitHub:** Version control and remote code repository. GitHub was preferred over UiA's Gitea for easier access.

4.2.3 Product and Report Creation

The following tools were used to facilitate product creation and report writing:

- **Figma:** Figma was used to create visual templates for how the mobile application should look and be experienced.
- **Android Studio with Expo App:** Android Studio was used to emulate an Android telephone so that the team could test the implemented solution as it was developed. The Expo App could also be used on physical mobile phones for the same purpose.
- **ChatGPT:** ChatGPT was utilized to ensure grammatical correctness of this report and support the team whilst producing the codebase for the mobile application.

4.3 Project Timeline

The project was planned to span 13 weeks. Figure 3 shows the distribution.

	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13
Idea Phase													
Planning													
Core Development													
Advanced Feature Development													
Final Adjustments													
Testing and Debugging													
Reporting													

Figure 3: Project timeline overview

4.3.1 Idea Phase

Problem domains were ranked and refined through group brainstorming, leading to the final concept. See Appendix P or Project Brainstorm.

4.3.2 Planning

Activities included creating a use case model, entity model, user stories, and wireframes. User stories were scored by effort and priority to define the MVP. Section 7.1 elaborates on the entity model.

Account and Login

AL1- As a parent, I want to create an account to access the app's features.

- **Priority: 8 Difficulty: 3**

Figure 4: User story example with effort and priority scores

4.3.3 Core and Advanced Development

The MVP features were implemented during the core phase, followed by advanced features in a later phase. Tasks were tracked in Taiga, and weekly Scrum meetings ensured an updated backlog and sprint creation.

4.3.4 Final Adjustments

The final week included bug fixes, polishing, gathering required documentation as provided in project delivery checklist, and publishing the app to Google Play.

4.3.5 Testing and Debugging

Testing and debugging took place over several weeks in parallel with the implementation. While the original plan was to rely on automated testing, technical challenges led us to instead run individual manual tests in Maestro. Firebase was updated between each test to ensure the necessary test data was available. A summary of test results is provided in chapter 11.

4.3.6 Reporting

The project report was written as the project developed, continuously being updated and edited by team members as they completed their assigned tasks.

4.4 Roles and Responsibilities

The team has worked successfully together before, so the process was done without a formal leader, but members with prior coordination experience naturally continued those responsibilities.

The team did, however, have two assigned positions:

- **Git Master:** Responsible for version control including performing merges after the end of a scrum sprint.

- **Scrum Master:** Responsibilities included managing the scrum backlog, ensuring that the team has a tidy and reliable work environment on Taiga and defining the scope of new Scrum sprints.

4.5 Research

4.5.1 Technology Stack

Relevant documentation and articles were reviewed to evaluate tools within four categories: frontend, backend, database, and hosting. This helped identify technologies that aligned with the project's goals and constraints.

4.5.2 Competitive Analysis

To understand the existing market, similar apps such as Gimi, Ukelønn, samboer.app, and neatkid were tested and compared. The evaluation considered usability, available features, and pricing models to identify gaps and opportunities for the project.

4.5.3 UI/UX Design

Both academic and industry sources were consulted to gather best practices in user experience and interface design. Some focus was placed on design for children. Key takeaways were applied throughout the development process to support usability and visual appeal.

5 Research Results

This chapter presents findings from our research on technology choices and user experience principles, as introduced in Chapter 4. These findings guided implementation decisions throughout the project.

5.1 Technology Stack: Frontend

Several cross-platform frameworks were evaluated based on platform support, language, documentation, backend compatibility, animations, and performance. Figure 5 presents a visual summary.

	ReactNative	Xamarin	.NET MAUI	Flutter	Apache Cordova	Ionic
Native or Cross Platform	Cross-platform	No support after 01.05.24. Recommended to use .NET MAUI instead	Cross-platform	Cross-platform	Cross-platform, web based	Cross-platform, web based
Language	JavaScript		C# and XAML	Dart	HTML, CSS, JavaScript	HTML, CSS, Javascript
Documentation/Support	Easy to understand, tutorial-like documentation provided by Meta Open Source		Well documented, easy to understand tutorial-like. Support provided by Microsoft	Well documented, easy to understand. Support provided by Google	Decent documentation, supported by The Apache Software Foundation	Well documented, easy to navigate.
Compatibility with backend	Excellent. Works well with REST APIs, GraphQL, Firebase, and third-party backends		Ideal ASP.NET integration	Supports integration with a variety of backends.	Supports integration with a variety of backends	Supports integration with a variety of backends
Animations and Sounds	Good range with libraries like Animated and react-native-sounds		Animation class with room for customization. Plugins for sound-effects	Excellent. Supports frameworks like Lottie and Flare, as well as widgets. Also supports games.	Uses Web based animations, recommended to use CSS animations. Plugins for sound	Uses Web Animations API and CSS animations. Less ideal for advanced animations
Performance	Very good. Uses native components.		Good performance, but slightly worse than ReactNative, but has support for Unit testing to improve performance of app	Excellent. Compiles Dart into native machine code. Can make complex apps without sacrificing perf	Uses WebView to render, thus slower but can be improved by third party plugins	Uses webView and JIT to render. Similar to ReactNative. Utilises less CPU when in use

Figure 5: Overview of frontend frameworks evaluated

React Native

React Native is a cross-platform framework by Meta for building native-like apps using JavaScript or TypeScript. It is well-documented and widely supported by the developer community.

It integrates smoothly with backends like REST APIs, GraphQL, and Firebase [18, 2], and supports advanced animations through tools like Animated API, react-native-reanimated, and Lottie [44, 50]. Performance is generally strong, though some overhead may occur due to the JavaScript bridge.

Xamarin

Xamarin is a cross-platform mobile development framework by Microsoft using C#.

It is no longer officially supported after May 2024, and Microsoft recommends transitioning to .NET MAUI [36].

.NET MAUI

.NET MAUI is the successor to Xamarin, enabling native app development for Android, iOS, macOS, and Windows using C# and XAML. It is well-documented and backed by the Microsoft ecosystem [35, 34].

It integrates well with ASP.NET Core and Azure, and supports improved animations, sound plugins, and unit testing. Performance is solid but slightly below React Native and Flutter due to additional rendering layers [57].

Flutter

Flutter is Google’s open-source toolkit for building natively compiled apps using Dart. It delivers high-performance UI through the Skia engine and supports advanced animations via libraries like Lottie and Flare [24, 51].

It integrates well with backends such as Firebase, REST APIs, and GraphQL, and compiles directly to native code for excellent performance.

Apache Cordova

Apache Cordova enables cross-platform apps using HTML, CSS, and JavaScript within a WebView. While easy to use, its performance lags behind native frameworks due to WebView reliance [48].

It supports any backend with HTTP APIs and handles animations via standard web methods, with CSS transitions preferred for better performance [7].

Ionic

Ionic builds on Cordova and modern frameworks like React or Angular. It slightly improves performance with optimized WebView use and modern APIs like the Web Animations API [61, 27].

It integrates well with REST APIs and real-time databases, but is less suited for demanding animations or real-time graphics.

5.2 Technology Stack: Backend

The backend of a mobile application handles data storage, security like authentication and authorization, and business logic [49]. Based on research into documentation quality, ecosystem, and suitability for our use case, several technologies were evaluated, see Figure 10.

	Django	Express.js	Firebase	Supabase
Language	Python	JavaScript	N/A (Backend-as-a-service)	N/A (Backend-as-a-service)
Authentication	Provides out-of-the-box authentication and authorization	No in-built authentication and authorization, but authentication and authorization strategies exist (use of third party libraries)	Provides a complete "drop-in" authentication system (including Google, Facebook authentication)/provides a SDK for adding various authentication methods	Built-in Authentication, Authorization, and User Management.
Documentation/Support	Easy to understand documentation provided by Django project	Available through OpenJS foundation	Documentation provided on official website	Documentation provided on official website (with "framework quickstarts")
Easy access to database	Comes with an ORM for easy data manipulation	No built in ORM but many compatible ORMs exist	Provides a NoSQL database based on a "documents" (key - pair) system	Provides a full PostgreSQL database
Compatibility with databases	PostgreSQL, MariaDB, MySQL, Oracle & SQLite	Compatible with many databases (by loading corresponding Node.js driver)	Provides inbuilt database	Provides a full PostgreSQL database

Figure 6: An overview of backend technologies

Django

Django is a high-level Python web framework known for rapid development and strong security features [10]. It includes built-in tools like an admin interface, URL routing, and form handling [47, 20], and is used by companies like Instagram and Spotify [43].

Express.js

Express.js is a lightweight web framework for Node.js used to create API endpoints [16, 12]. Its popularity ensures a wide range of third-party packages and community support [32].

Firebase

Firebase is a Backend-as-a-Service that handles authentication, database access, and push messaging out of the box [40, 52]. It allows faster development by removing the need to manage backend infrastructure.

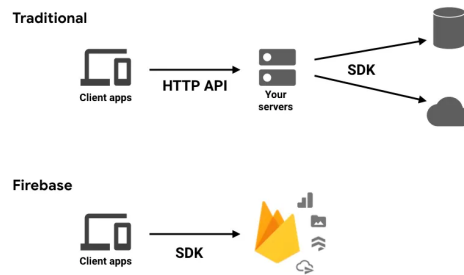


Figure 7: Comparison of traditional backend and Firebase [52]

Supabase

Supabase is an open-source alternative to Firebase that offers a relational SQL database and a more predictable pricing model based on data storage [53, 46]. It is reported to offer faster read/write operations than Firebase [19].

5.3 Technology Stack: Local Database

An “offline-first” architecture improves user experience by allowing the app to function without a constant internet connection [5]. According to Android developer documentation, such apps should remain usable without reliable network access, present local data immediately, and fetch data only under optimal conditions—such as when the device is charging or connected to WiFi [4].

To implement this architecture, several local database technologies were considered.

SQLite

SQLite is a lightweight embedded database that requires no setup or server, and is widely supported across platforms [21]. However, since it stores the entire database in a single file, it is more susceptible to corruption and security vulnerabilities [45].

Realm

Realm is a modern alternative, often positioned as a replacement for SQLite. It offers higher performance, built-in data encryption, and supports real-time synchronization with MongoDB Atlas, which includes automatic conflict resolution [28].

Couchbase Mobile

Couchbase Mobile is a NoSQL solution that provides an embedded database and supports bi-directional synchronization with a Couchbase server, either hosted or self-managed. It is designed for complex offline use cases and supports fine-grained sync control.

Firebase Realtime Database

Firebase Realtime Database, while cloud-based—can be used in offline-first scenarios by enabling local disk persistence. This allows the app to read and write data locally while offline, with automatic syncing once connectivity is restored [14].

5.4 Technology Stack: Remote Database and Hosting

The team decided that the need for researching remote databases and hosting options was unnecessary as there was a time constraint on the project, and the focus should lie on app user experience.

5.5 Competitive Analysis Results

As part of the research phase, a competitive analysis was conducted to explore existing applications aimed at helping parents and children manage chores and rewards. Four apps were analysed: *Gimi*, *Ukelønn*, *samboer.app*, and *neatkid*.

Gimi

Gimi focuses on financial education and saving. It requires children to have their own email address or access to their parent’s email to log in. Switching between parent and child profiles requires logging in and out. Children can suggest tasks via the app, but these must be approved by parents before becoming active. It is also possible to link the app directly to a bank account to facilitate money transfers through Gimi. Core features such as savings goals and most of the app’s quizzes and stories are locked behind a Gimi Pro subscription [23].

Ukelønn

Ukelønn offers a free solution where parents can assign both one-time and recurring tasks and define rewards such as money or activities. Photo verification can be requested upon task completion. The app includes a selection of icons for task categorization, and parents can send reminders via in-app notifications. Like Gimi, it requires separate accounts for parents and children, which can be cumbersome if a device is shared [8].

samboer.app

samboer.app helps manage household chores. While free to download, features like custom chores, chore values, and detailed statistics require a premium subscription (549 NOK/year or 99 NOK/month) [3]. Users can assign household roles (adult, teenager, child, or “person without a device”). Tasks completed by users without devices must be logged by someone else. The app emphasizes visual stats over monetary rewards [3].

neatkid

Neatkid is a chore and reward management app for children [39]. It requires the child to have access to their own device, but does not require a personal email or phone number. A verification code is generated when the parent adds a child, enabling access. The app is entirely subscription-based, with no free features. Pricing options include 39 NOK/month, 249 NOK/year, or a one-time fee of 599 NOK for lifetime access [39].

Key Insights from Competitive Analysis

The competitive analysis of *Gimi*, *Ukelønn*, *samboer.app*, and *neatkid* highlights differences in user access, features, and reward systems.

- **Access and User Accounts:**

Gimi and *Ukelønn* require email accounts for children, which may be impractical. *Neatkid* uses a verification code instead. *Samboer.app* supports users without devices by allowing others to register tasks for them.

- **Free vs. Paid Features:**

Ukelønn offers many features for free. *Gimi* and *Samboer.app* restrict core features to premium users, while *neatkid* is entirely paywalled.

- **Reward Systems:**

Ukelønn supports rewards like money or activities. *Gimi* uses stars for avatars. *Neatkid* offers similar features, but only via subscription. *Samboer.app* provides stats instead of rewards.

- **User Experience and Profile Switching:**

Gimi and *Ukelønn* require logouts to switch profiles. *Neatkid* simplifies this with codes, and *Samboer.app* lets users manage tasks for others without switching.

In summary, many apps require personal devices or restrict functionality behind paywalls. These insights guided our design priorities, especially around access, flexibility, and shared-device usability.

5.6 UI/UX Research Results

5.6.1 User Experience and User Interface

UX and UI are essential in mobile app development, influencing both usability and success. UI (User Interface) includes visual and interactive elements like buttons, menus, colours, and layout, aiming to make the app appealing and easy to use. UX (User Experience) concerns the overall interaction—how intuitive, efficient, and satisfying the app is [11].

It is crucial to consider the target audience and the problem the app solves. Research and analysis of similar apps offer useful insights. The design process typically begins with sketches and diagrams, followed by implementation [11].

Key UI elements, icons, typography, colours, images, and animations, affect how the app is perceived. Clear icons improve usability. Appropriate fonts and sizes support readability. Colours and layout create visual impressions, with colour psychology influencing perception [11]. Text should be at least 12 sp, and contrast between text and background should be a minimum of 4.5:1 [1]. Since users tend to skim, the most important messages should be placed at the beginning [58]. Images reinforce messaging, while animations enhance usability and satisfaction [11].

UX design focuses on the user. A well-designed app evokes positive emotions and supports intuitive navigation [11]. One should highlight key elements, ensure predictable navigation, and use established design patterns as a foundation [58].

Break down complex tasks into smaller steps and aim for fewer than three clicks to reach any feature [58]. Let users save progress and continue later, and minimize text input and prefill fields when possible. Apps should load quickly - if loading, show a spinner or similar indicator to avoid confusion [58]. Use language suited to the audience and improve it through feedback [11].

Modern users expect apps to be both visually appealing and easy to use. With clear goals, good planning, and audience insight, developers can deliver user-friendly, efficient apps. Great UX depends on thoughtful design, clear language, and intuitive flow [11].

5.6.2 Designing for Children

Child-centered design means addressing children’s specific needs and experiences. Children think more concretely than adults, requiring simpler interaction models [56].

Design for children should include audiovisual and sensory elements. Younger children need large, recognizable on-screen objects. Colourful buttons that stand out from the background are important. Menus should avoid submenus, and icons should be large and easily understood [30].

A virtual assistant—such as a friendly character—can guide children and make the experience more interactive [55]. Children often prefer assistants that match their gender, followed by animals or other fun characters [30].

Music and sounds significantly enhance children’s engagement. Sound effects and music shape how children interact with apps [30].

Tasks should be age-appropriate. If too hard, children may lose interest [30]. Immediate, positive feedback helps maintain engagement. Rewards like stars or points, ideally paired with sound or visual celebration, improve the overall experience [56, 30].

6 Requirements

This section lists the main functional requirements needed to create the Minimum Viable Product (MVP) for the application. These requirements comes from user stories and have been prioritized based on the importance and development difficulty using the Fibonacci scale:

- **1-3: Could-have** - Desirable features that enhance the product but are not essential for the MVP.
- **5: Should-have** - Important features that provide significant value but can be postponed if necessary.
- **8: Must-have** - Essential features that are critical to fulfilling core user needs and delivering the MVP.

The tables below show only the requirements selected for the MVP. The full overview over all requirements, both functional and non functional, can be found in Appendix H, as well as directly here: *Requirements*.

6.1 Functional Requirements

6.1.1 App Access

Table 1: App Access Requirements

ID	Requirement Description	Priority	Difficulty
AL1	Parent create an account to access the app's functionalities	8	3
AL2	Parent create a household to connect family under one account	8	3
AL3	Parent log in with username and password to access the app's functionalities	8	5
AL7	Parent log out of the app	8	8
AL8	Parent log out of parent user	8	3

6.1.2 Task Management

Table 2: Task Management Requirements

ID	Requirement Description	Priority	Difficulty
TM1	Parents are able to view all active tasks	8	8
TM2	Parents are able to create tasks and assign them to one ore more children so that the children can view them in their profiles	8	3
TM3	Parents are able to edit when a task should be completed, either one-time or recurring on selected weekdays	8	5
TM4	As a parent, I want to edit points on tasks so children know how much each task is worth	8	2
TM9	Parent are able to edit tasks, such as updating descriptions	8	2
TM10	Parent are able to delete tasks that are no longer relevant	8	2
TM12	Parent are able to see tasks that have been marked as done and are awaiting approval	8	3
TM13	Parent are able to approve completed tasks so that children receive their rewards	8	5
CFT1	Children are able to mark tasks as completed	8	5

6.1.3 Reward System

Table 3: Reward System Requirements

ID	Requirement Description	Priority	Difficulty
RS1	Parent are able to view all available rewards in order to know that options are available for children	8	3
RS2	Parent are able to add new rewards that children can "buy" with their earned points	8	3
RS3	Parent are able to edit the point cost for each reward	8	3
RS4	Parent can see which rewards their children have purchased	8	3
RS5	Parent can mark a reward as used once the child has received it	8	3
RS7	Parent are able to delete rewards that no longer are relevant	8	3
CFR1	Children are able to view a list of available rewards that they can purchase with their points	8	3
CFR2	Children are able to see which rewards they already have earned	8	3

6.2 Non-Functional Requirements

Table 4: Non-Functional Requirements

ID	Requirement Description
NFR1	Simple interface - The app should be easy for children (ages 6+) to use while still being practical for parents.
NFR2	Age-appropriate design - Visually engaging and easy to understand for children, but also organized for parents.
NFR3	Quick setup - Users should be able to create an account and understand the basics in under 5 minutes.
NFR7	GDPR compliance – User data must be handled in accordance with privacy regulations.
NFR8	Role-based access: Parents should have admin privileges, children should not.

7 Solution

7.1 Entity Model

The entity model maps out the interaction between the key entities within the proposed mobile application. Key features of the entity model include:

- **Parent:** Parents can be considered as "admin users", creating new child, task and reward objects and managing their connections.
- **Child:** A child is also a user but with restricted functionality that focuses on completion of tasks and purchase of rewards
- **Task:** Each task is associated with a specific child by including a `childId` attribute in the `Task` object. This ensures that a task always belongs to one particular child from the moment it is created.
- **PurchasedReward:** Rewards can be purchased multiple times by different children. To represent this many-to-many relationship, an association object called `PurchasedReward` is used, containing both `childId` and `rewardId`, along with additional information such as the time of purchase.

See Figure 8 for a visualization of the entity model or find a high resolution version in Appendix M, as well as directly here: *Entity-Model.svg*.

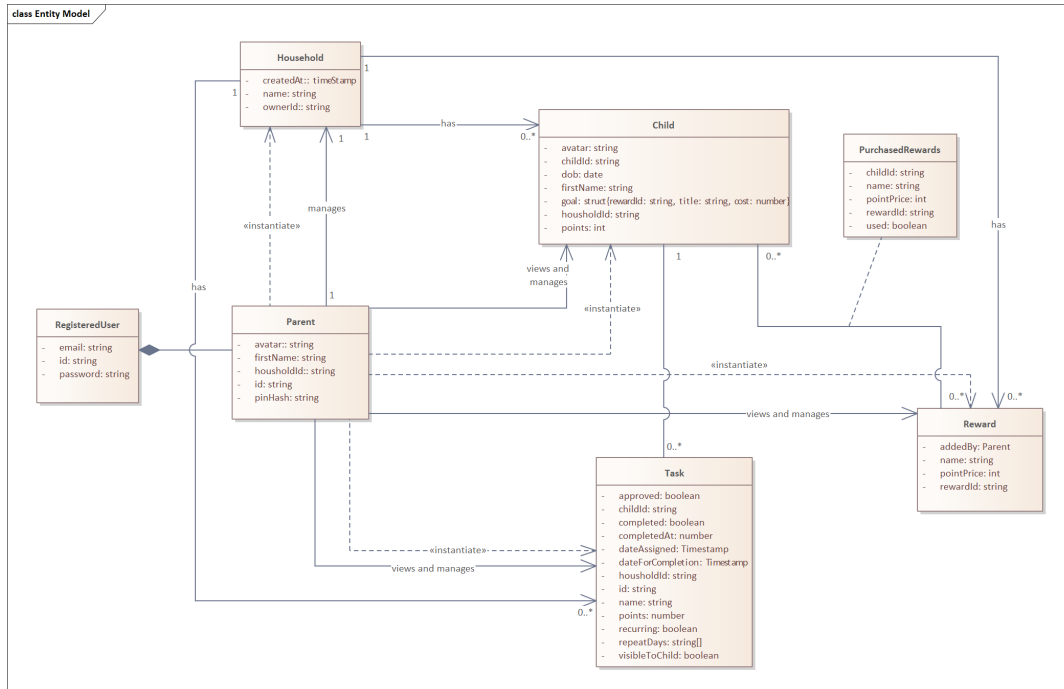


Figure 8: Entity model for FortGjort

7.2 Architecture

The product will utilize a Component-Based architecture. A component is "a group of related functionality that resides behind a nice and clean interface" [13]. The solution will consist of components working together to provide the required functionality.

The characteristics of a Component-Based architecture include [6]:

- **Reusability**: Components can often be reused in multiple parts of the application.
- **Replaceable**: A component can be swapped with a similar one.
- **Not context specific**: Designed to work independently of context.
- **Extensible**: A component can be built from smaller ones without changing their internals.
- **Encapsulated**: Internal logic and data are hidden; other components access it via a defined interface.
- **Independent**: Components have minimal dependencies on each other.

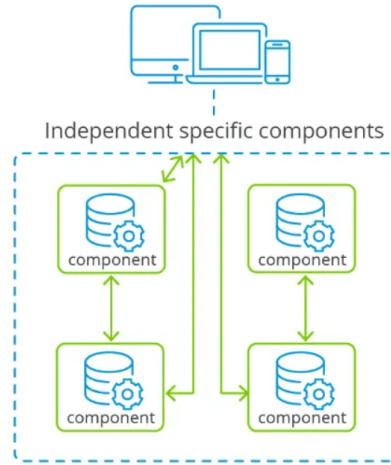


Figure 9: Component based architecture[29]

7.3 App Structure and Routing

Based on insights gathered during the research phase (see subsection 5.6), different navigation strategies were chosen for parents and children to optimize the user experience for each group.

For the parent interface, a bottom tab navigation structure was implemented. This decision was based on its familiarity and clarity, making it easy to switch between key functionalities such as tasks, rewards, and settings. Each tab contains a dedicated stack navigator, allowing the user to navigate deeper into related subpages while maintaining context within the selected tab.

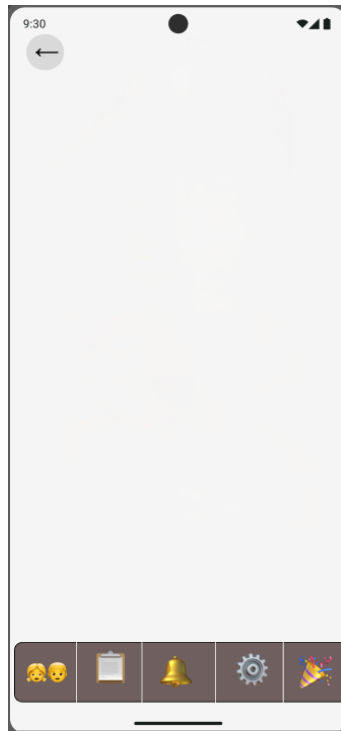


Figure 10: Conceptual views of parent navigation design

The child interface was designed with simplicity and cognitive load in mind. Research suggested that bottom navigation bars are less suitable for younger users. Instead, a floating action button was introduced, providing access to a pop-up menu with large, clearly labeled buttons for core actions: Home, Tasks, Rewards, and Logout.

This floating menu is persistently available, enabling intuitive navigation from anywhere in the child's view. The implementation focused on accessibility, minimal distractions, and quick interactions, aligning with UI/UX recommendations for child-friendly apps.

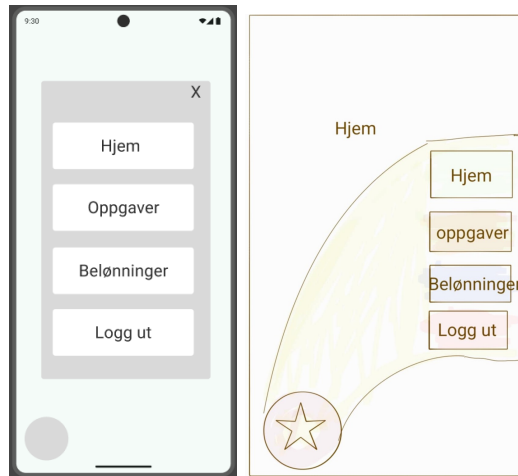


Figure 11: Conceptual views of child navigation design

7.4 Product

Figma designs were created to align with the requirements for a Minimum Viable Product. The main features of the solution are presented below, grouped according to core functional areas such as login and administration, the task system, and the reward system.

All views, both the implemented and not implemented, can be found in Appendix I, as well as directly here: *Figma Views*.

7.4.1 App Access

Before users can access the application, they must register by providing their name, email address, and password, as specified in user story AL1. Upon successful registration, a parent user account is created.

TITTEL

Fornavn

Epost

Passord

Gjenta passord

Fullfør Avbryt

Figure 12: View of solution:
User registration

TITTEL

Epost

Passord

Logg inn

Glemt passord Registrer deg

Figure 13: View of solution:
Log in

The first time a parent logs in (AL3), they are prompted to create a household (AL2). As part of the household creation process, the user is required to set a household PIN.

Legg til husstand 🏠

Bli med i husstand 🏠

Figure 14: View of solution:
Create household

Husstandsnavn

Husstands pin

Fullfør Avbryt

Figure 15: View of solution:
Create new household

If the parent already has an account and a registered household, they will be directed to the household home screen upon logging in.

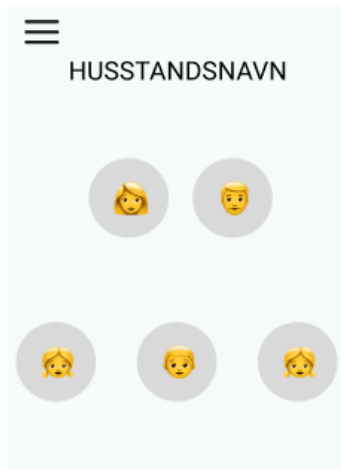


Figure 16: View of solution: Household home screen

7.4.2 Tasks Management

In FortGjort, parents can create tasks which includes choosing the amount of points a child receives upon the completion of the task and assigning the task to a child/children, thus fulfilling the requirements detailed in TM2 - Create and assign tasks.

When creating a new task, the parent can choose whether it should be visible to the child. If set to visible, the task will appear for both the parent and the child. If set to hidden, only the parent will be able to see it.

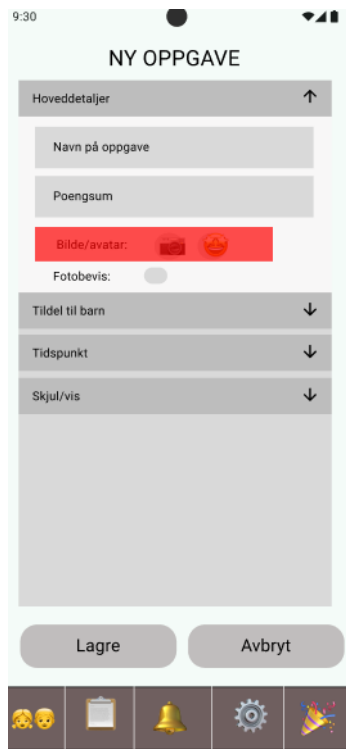


Figure 17: View of solution: Part of the process of a parent creating a new task, adding task name and points

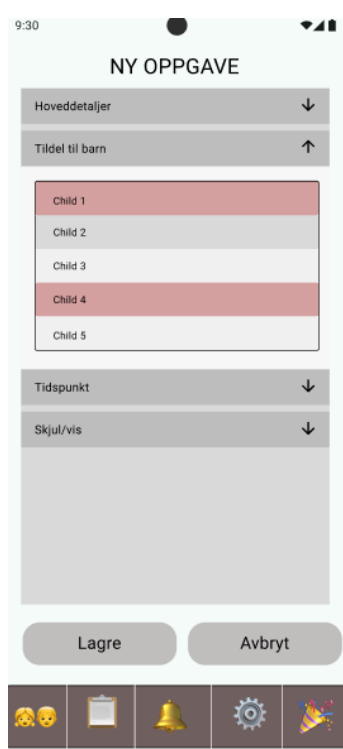


Figure 18: View of solution: Part of the process of a parent creating a new task, assigning it to one or more child(ren)



Figure 19: View of solution: Part of the process of a parent creating a new task, choosing if it should be visible to the child(ren) or not

Tasks can be set as either one-time or recurring. For one-time tasks, the parent can select a specific due date from the calendar. For recurring tasks, the parent can choose the days of the week on which the task should be completed.

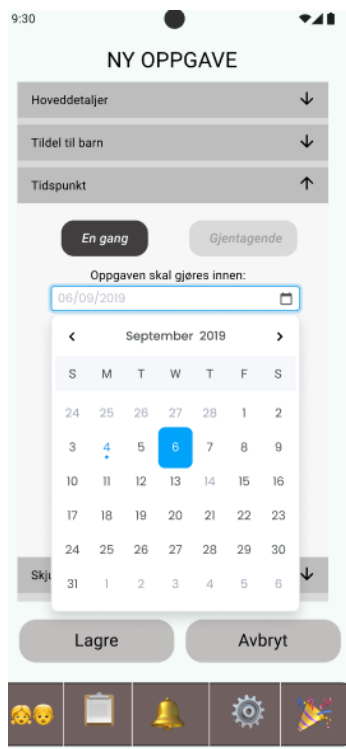


Figure 20: View of solution: Part of the process of a parent creating a new one-time task, selecting date for completion



Figure 21: View of solution: Part of the process of a parent creating a new recurring task, selecting weekdays the task should be completed

Upon the child completing a task, a parent can evaluate the task so that, if the task is successfully completed, the associated points are added to the child's account. Figures 22 and 23 illustrate this part of the solution, showing how TM13 - Approve completed tasks, is fulfilled.



Figure 22: View of solution:
A parent can view the task
that a child has completed

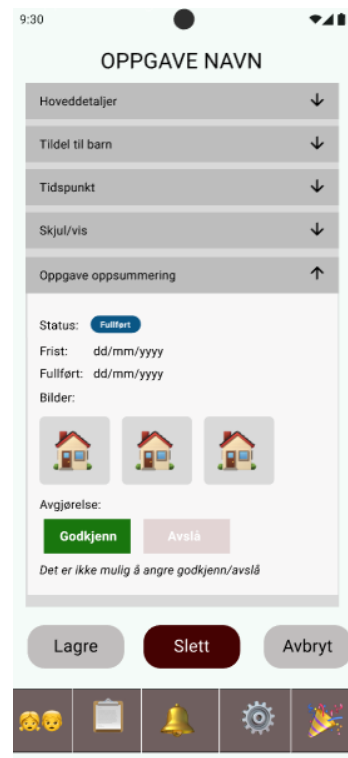


Figure 23: View of solution:
A parent can approve a task
that a child has completed

Tasks can be deleted if they have either not yet been completed by the child or have already been approved by the parent. This allows the parent to manage and clean up tasks that are no longer relevant or were added by mistake. Figures 24 and 25 illustrates this part of the solution, showing how TM10 - Delete tasks, is fulfilled.



Figure 24: View of solution: A parent can delete a task that is not completed by a child

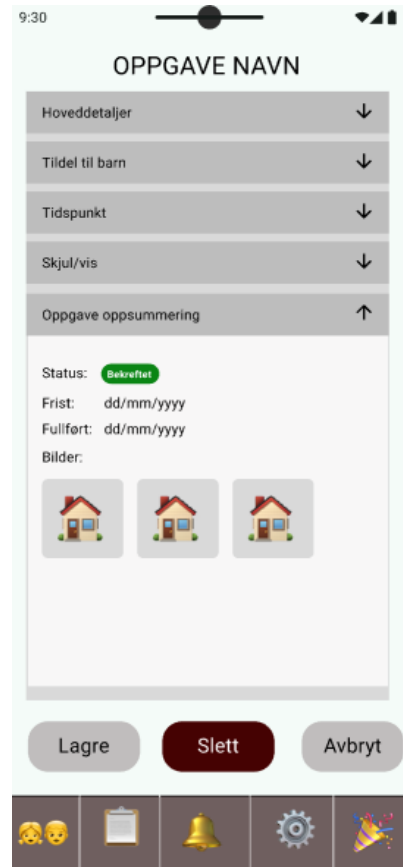


Figure 25: View of solution: A parent can delete a task that is completed and approved

See my tasks as child

When a child selects their profile from the household home screen, they are taken to a simplified, child-friendly interface. From this view, they can navigate to the “Oppgaver” section, where all tasks assigned to them by a parent are presented in a visually intuitive format as shown in Figure 26.

Tasks can then be marked as completed as illustrated in Figure 27, thereby fulfilling requirement CFT1.



Figure 26: View of solution: A child may view their assigned tasks by clicking on "oppgaver" in the menu after entering their child profile.



Figure 27: View of solution: A child can mark the task as completed or uncompleted

7.4.3 Reward System

Parents can create new rewards to be placed in a "reward bank", specifying whom the reward should be shown for, or if the reward should be shown at all.

Figure 28 illustrates how parents can see all rewards stored in the "reward bank" thus fulfilling RS1. Figure 29 illustrates how parents can add new rewards thus fulfilling RS2.

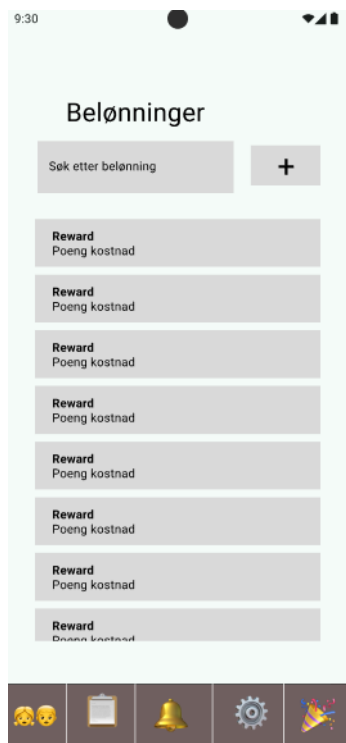


Figure 28: View of solution:
List of available rewards

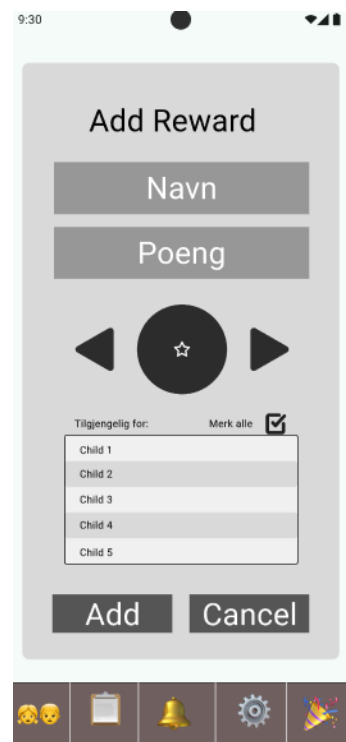


Figure 29: View of solution:
A parent can add a new re-
ward to the reward-bank

To fulfil CFR1, a child can view available rewards to "purchase" with the points they accumulate through completing tasks as shown in Figure 30. If they have enough points, they can select and purchase a reward from this list, as shown in Figure 31. Once a reward has been purchased but not yet received in real life, it is placed in the child's backpack (CFR2), as illustrated in Figure 32.



Figure 30: View of solution: List of available rewards



Figure 31: View of solution: Child buy reward



Figure 32: View of solution: Purchased but not received rewards

A parent is also able to see which rewards a child has, and mark the reward as used or unused. This feature is illustrated in figure 34.

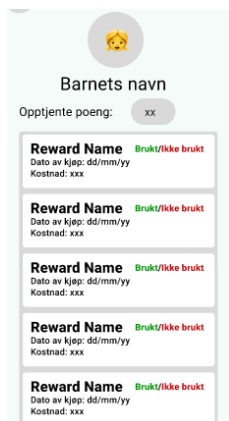


Figure 33: View of solution: Rewards waiting for approval

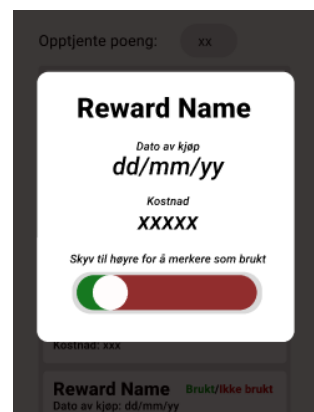


Figure 34: View of solution: A parent can mark a reward as used

8 Technical Background

8.1 Choice of Technologies

8.2 React Native

React Native is a framework that allows the development of mobile applications for both Android and iOS using a shared codebase written in JavaScript and React [22]. It makes it easy to build apps that look and behave like true native applications.

Instead of rendering web content, React Native translates its components into actual Android or iOS elements. This ensures that the application appears and functions like a standard Android or iOS app in terms of both design and performance [37].

Although one of React Native's main advantages is code reuse across platforms, the framework also supports platform-specific customisation when needed [41].

In applications built with React Native, build-in React features such as `useState` and `useEffect` can be used to manage state and define logic that should run when a component is loaded [22]:

To build the user interface, pre-made components provided by React Native can be used. These components serve the following purposes:

- **View**: used as a container for layout
- **Text**: displays text content
- **TouchableOpacity**: creates interactive, pressable elements
- **ActivityIndicator**: shows a loading spinner

These components are built into React Native and are rendered as standard Android components [37].

8.3 Firebase

Firebase is a cloud-based platform that provides developers with a comprehensive suite of tools for building, testing, and managing web and mobile applications. It supports the entire application lifecycle and is structured around four key areas [33]:

1. Development
2. Growth
3. Monetization
4. Analytics

One of the core components of Firebase is Firestore, a scalable NoSQL database that enables real-time data synchronization. This allows developers to create responsive and dynamic applications without the need to maintain their own servers [33].

Firebase also offers robust authentication services through Firebase Authentication. This makes it straightforward to implement user login systems using email and password, as well as third-party providers such as Google and Facebook [33].

The platform supports multiple development environment, including Android and iOS, and its services scale based usage. Firebase also integrates seamlessly with external tools like GitHub, making it a flexible and efficient solution for modern application development [33].

Firebase can be integrated in a project by getting initialized with a configuration object that connects the application to the correct Firebase project. The setup ensures that the application can securely handle user authentication and interact with a cloud-based database in real time.

8.4 Expo

Expo is an open-source platform built on top of React Native, designed to simplify the development, testing, and deployment of mobile applications [31]. The platform offers a so-called "managed workflow", where tools and configurations are pre-configured. This allows developers to focus on the core application logic rather than spending time setting up native code and build environments [31].

A central feature of Expo is support for over-the-air (OTA) updates, which allows developers to push new versions of the app directly to user's devices - without having to resubmit the app to the App Store or Google Play [31].

The Expo CLI tool is used to create and manage projects, while the Expo Go app makes it easy to test the application in real time - either using an emulator on the computer or by scanning a QR code to test the app directly on a mobile device. This enables immediate feedback and testing without the need to rebuild the entire project [31].

8.5 Maestro

Maestro is a simple and powerful end-to-end testing platform. Maestro provides a way of testing without having to compile the tests beforehand. Tests are defined in a yaml file, and interactions are declared as a flow of actions.

Maestro also has built-in tolerance to delays, so that writing tests can focus on simulating user actions. A simple test to illustrate Maestro's simplicity is provided:

```

1  appId: com.anonymous.choresapplication
2  name: Create Account
3  # description: A simple flow to test the app's launch and welcome screen.
4  clearState: true
5  onFlowStart:
6    - launchApp
7  onFlowComplete:
8    - stopApp
9  ---
10 - tapOn:
11     id: "password_input"
12 - inputText: "Testing123."
13 - tapOn: "Login"
14 - assertVisible: "You are logged in"
15

```

8.6 Routing with Expo Router

Expo Router is a file-based routing library for React Native apps built with Expo. It builds on React Navigation but simplifies navigation by mapping the file structure inside the `app/` directory directly to routes[26].

Each file in `app/` represents a screen, and folders wrapped in parentheses (e.g., `(tabs)`) act as route groups used for organizing routes without affecting the URL[15]. Files named `index.tsx` serve as default routes for their folders, while `app/index.tsx` defines the app's main entry point.

The `_layout.tsx` file structures navigation within its folder using navigators like `<Stack />` or `<Tabs />`. The root layout (`app/_layout.tsx`) replaces the traditional `App.tsx` and is used for global setup like splash screens or font loading[15, 38].

This approach supports nested stack and tab navigators, reduces boilerplate, and results in more maintainable and declarative navigation structures.

8.7 Expo Crypto

`expo-crypto` is a universal library that provides cryptographic functionality for React Native applications. It is designed to mimic the Node.js core `crypto` module and works seamlessly across platforms. The library supports hashing and generation of cryptographically secure random values and includes support for standard algorithms such as SHA-1, SHA-256, SHA-384, and SHA-512[9].

This method takes a hashing algorithm (e.g., SHA-256) and a text string, returning a cryptographic hash as a string.

```
Crypto.digestStringAsync(algorithm, data, options?)
```

By default, the result is returned in HEX format. Expo Crypto can be used to securely hash PIN codes before storing them in Firestore, ensuring that plaintext PINs are never saved or transmitted. On web platforms, all methods require a secure origin (HTTPS or localhost). The library is optimized for use in Expo-based projects[9].

9 Security

In this project, a security analysis was conducted based on the OWASP Mobile Application Security Verification Standard (MASVS). MASVS is a well-established industry standard that provides developers with clear guidelines for building secure mobile applications[65]. The analysis covered key areas such as data storage, authentication, network security, and protection against code tampering.

The results of the MASVS-based security analysis indicate that the application has a solid security foundation. Core areas such as secure data storage, encryption, and authentication are well addressed through the use of Firebase and secure coding practices. Some controls, such as resistance to reverse engineering and enforced updates, are not currently implemented, but are considered low risk due to the app's scope, user group, and threat model. Other areas, like network security and privacy, are partially fulfilled, with improvements planned for future development.

A full version of the analysis is included in Appendix J, as well as directly here: *Security Analysis*, while a summarized overview of the results is presented in the table below.

MASVS Category	Control Areas (excerpt)	Status	Comments
STORAGE	Secure local storage, avoid leaks	Fulfilled / Low risk	All sensitive data is encrypted in Firebase. Images are only temporarily cached and deleted after upload. Automatic backup is considered and will be handled in future development.
CRYPTO	Use of encryption and secure key management	Fulfilled	All data is encrypted in transit (HTTPS) and at rest (Firebase). A 4-digit user code is encrypted locally using expo-crypto; keys are not stored. PINs are hashed (SHA-256) before storage and never saved in plaintext.
AUTH	Authentication, local authentication, and extra protection for sensitive actions	Fulfilled	Firebase Authentication ensures secure external login. A local 4-digit PIN protects the parent section and sensitive features. Token-based access control and PIN validation follow best practices.
NETWORK	Secure network traffic, use of HTTPS and certificate pinning	Partially fulfilled	All traffic uses HTTPS via Firebase. Certificate pinning is not implemented, but considered unnecessary for Firebase. Will be reassessed if custom APIs are added.
PLATFORM	Secure interaction with mobile platform and other apps	Fulfilled / Not directly relevant	No use of IPC or WebViews. Camera access requires consent. Sensitive data is PIN-protected and not exposed to other apps.
CODE	Code quality, secure libraries, input validation, update routines	Partially fulfilled	The app uses secure and up-to-date components. Input is handled safely. Forced updates are not implemented, but planned as a future improvement.
RESILIENCE	Resistance to reverse engineering, tampering, and analysis	Not implemented, but low risk	Not implemented, but considered low risk based on the threat landscape and the nature of the app's content.
PRIVACY	Privacy, user consent, data minimization, and control	Partially fulfilled	The app collects only necessary data with consent. A privacy policy and user-initiated data deletion features are planned. No third-party SDKs or tracking are used.

Figure 35: Overview - Security analysis

10 Implementation

10.1 Entity Model

The entity model was implemented in the project using Firebase Firestore as the underlying cloud-hosted NoSQL database. Data was organized in a hierarchical structure with top-level collections and subcollections nested within documents. This design reflects the natural hierarchy between households, users, and their related data.

- **households**: This top-level collection stores documents representing each household. Each document contains metadata such as the name of the household, creation timestamp, and a reference to the owning parent via the `ownerId` field. Within each household document, the following subcollections are defined:
 - **children**: Stores documents for each child in the household. Each child document includes fields like `firstName`, `dob`, and `goal`.
 - * **purchasedRewards**: This was implemented as a subcollection under each `child` document. Each purchased reward document contains fields such as `rewardId`, `name`, `pointPrice`, and a `used` flag.
 - **rewards**: Contains all reward objects created within the household, each with a `name`, `pointPrice`, and `addedBy` field referencing the parent user.
 - **tasks**: Tasks are stored per household and reference the child they are assigned to via the `childId` field.
- **parents**: This top-level collection contains documents for all registered parent users, storing user profile data and linking to their respective households via the `householdId`.

Data Modeling and Firestore Integration

Each Firestore entity was mirrored in the codebase using TypeScript interfaces. An example of the `Child` model is shown below:

```
1 export interface Child {
2   id: string;
3   firstName: string;
4   householdId: string;
5   avatar: string;
6   dob: string;
7   points: number;
8   goal?: {
9     rewardId: string;
10    title: string;
11    cost: number;
12  };
13 }
```

Listing 1: The Child model

Firestore operations such as create, update, retrieve, and delete were encapsulated within dedicated `services.ts` files. These service functions abstract away direct database calls and provide an interface for interacting with Firestore. Below is an example of a function used to retrieve a single child document by its ID:

```
1 export const getChildById = async (householdId: string, childId: string): Promise<Child> => {
2   const childRef = doc(db, collections.childDocPath(householdId, childId));
3   const snap = await getDoc(childRef);
4   if (!snap.exists()) {
5     throw new Error('Barn ikke funnet');}
6   return {
7     id: childId,
8     ...snap.data(),
9   } as Child;
10 };
```

Listing 2: Retrieving a single child document by its ID

This structure promotes modularity, testability, and separation of concerns within the application code.

10.2 Architecture

The component-based architecture described in the solution phase was implemented through a modular and feature-oriented folder structure in the project directory. The application was organized to promote encapsulation, reusability, and clear separation of concerns.

All core functionality was grouped by domain under the `features` directory in the project. Each feature—such as `child`, `parent`, `task`, `reward`, and `purchase` was

implemented as its own module, containing all the necessary files related to that domain.

Within each feature folder, the following structure was used:

- **components:** Reusable UI components specific to the feature (e.g., modals, forms, and menus).
- **models:** TypeScript interfaces that define the structure of data related to the feature.
- **screens:** User interface screens connected to the navigation system.
- **services:** Functions for communicating with Firestore, encapsulating data operations such as fetch, add, and update.
- **styles:** Local styling files scoped to the feature's UI elements.

For example, the `features/child` directory includes all functionality related to children, including screens for viewing tasks, setting goals, and redeeming rewards, as well as models and Firestore services for retrieving child data.

This modular structure aligns with component-based architecture principles by making each feature self-contained, maintainable, and reusable across the application.

10.3 App Structure and Routing

Parent Navigation: Tabs, Stacks and Drawer

The parent section of the app is structured using a tab-based layout defined in `app/(parent)/(tabs)/_layout.tsx`. This layout uses the `<Tabs>` component from Expo Router to define five primary tabs available to parent users: *Profile*, *Tasks*, *Rewards*, *Notifications*, and *Settings*.

Each tab corresponds to a subdirectory, such as `profile/`, `settings/` or `tasks/`. These directories may contain multiple nested screens. For instance, the `profile/` tab includes screens for viewing and editing a child profile, adding new children, and viewing individual task and reward histories.

To handle these deeper navigational flows within a tab, each tab folder uses a nested layout file `_layout.tsx` that returns a `<Stack>` component. This enables stack-based navigation within a tab — e.g., pushing a screen such as `edit-child.tsx` on top of the current `profile/index.tsx`. These stack navigators provide consistent headers and back-navigation within each tab.

Additionally, the parent section uses a drawer layout for account-level actions, specifically logout. This drawer is configured in the parent root layout (`app/(parent)/_layout.tsx`), using the `<Drawer>` component from Expo Router.

The drawer contains a logout button that navigates the user back to the household entry point using `router.replace('/(household)/home')`. A simplified example of file-based routing in Expo Router is shown in Figure 36. The structure illustrates how the parent's profile view is organized hierarchically, with `_layout.tsx` files used to define navigation relationships between screens.

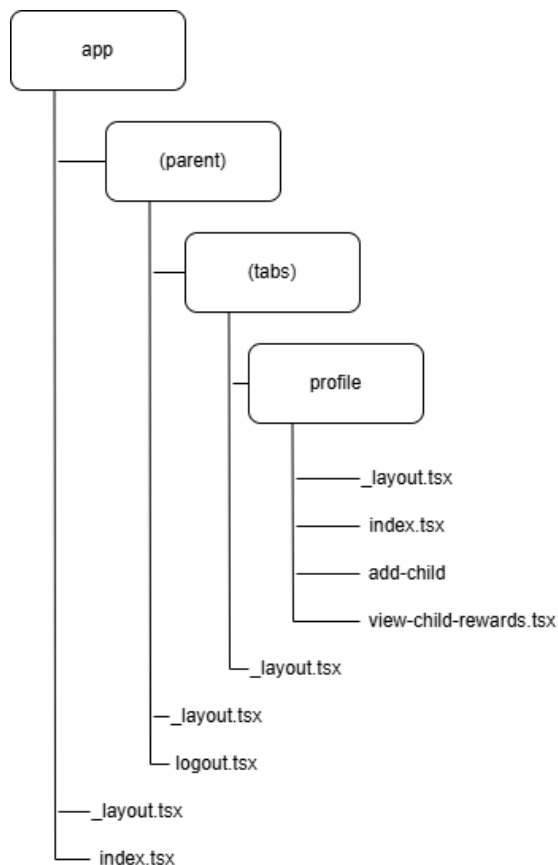


Figure 36: Simplified visualization of file-based routing in `app/(parent)/(tabs)/profile`



Figure 37: The tab bar in parent view

Child Navigation and Floating Menu

The child section of the application is implemented as a tab-based layout, defined in the file `app/(child)/(tabs)/_layout.tsx`. This layout sets up the tab navigator using Expo Router's `<Tabs>` component. Notably, the actual tab bar is hidden from the UI using the option `tabBarStyle: { display: 'none' }`, meaning navigation is not controlled by standard visible tabs.

Instead, navigation is handled through a custom "floating menu", implemented as a React component called `FloatingMenu`. This menu is rendered alongside the tabs layout and appears as an animated star button in the lower part of the screen. When the user taps the star, the menu expands with animated transitions and curved background, and presents custom navigation buttons such as “Hjem”, “Oppgaver”, “Belønninger” and “Logg ut”.

Below is a shortened example of how the floating menu logic was implemented using React Native:

```
1 {menuVisible && (  
2   <>  
3     <CurvedMenuBackground />  
4     <View style={styles.menuContainer}>  
5       <MenuItem title="Home" icon="home" onPress={() => router.push('/home')} />  
6       <MenuItem title="Tasks" icon="checkmark-done" onPress={() => router.push('/tasks')} />  
7       <MenuItem title="Rewards" icon="gift" onPress={() => router.push('/rewards')} />  
8       <MenuItem title="Logout" icon="log-out" onPress={() => router.replace('/home')} />  
9     </View>  
10  </>  
11 )}
```

Listing 3: A shortened example of how the floating menu logic was implemented using React Native

Each ‘MenuItem’ includes navigation logic using the ‘router.push()’ function, as well as dynamic props such as icons and colors. The actual implementation also includes, animation callbacks, and accessibility support, which have been omitted in this example for clarity.

This solution provides a more playful and child-friendly user interface with large flat buttons, while still leveraging the file-based routing of Expo Router. The implementation eliminates the need for reading proficiency and avoids nested submenus which can be confusing for children.

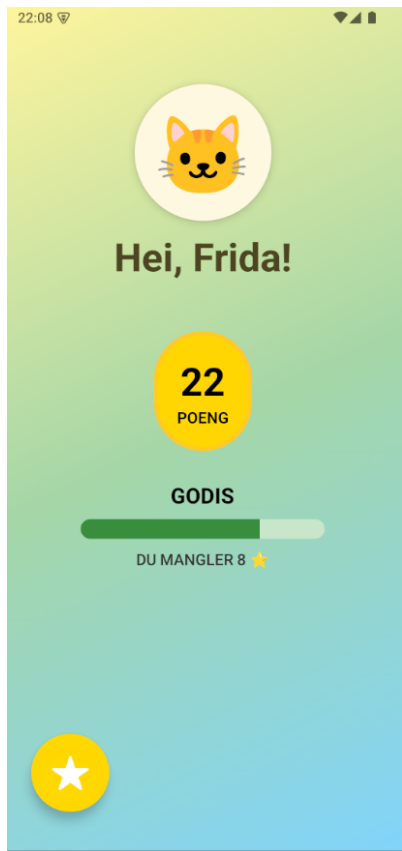


Figure 38: Child homescreen with the menu button in left corner

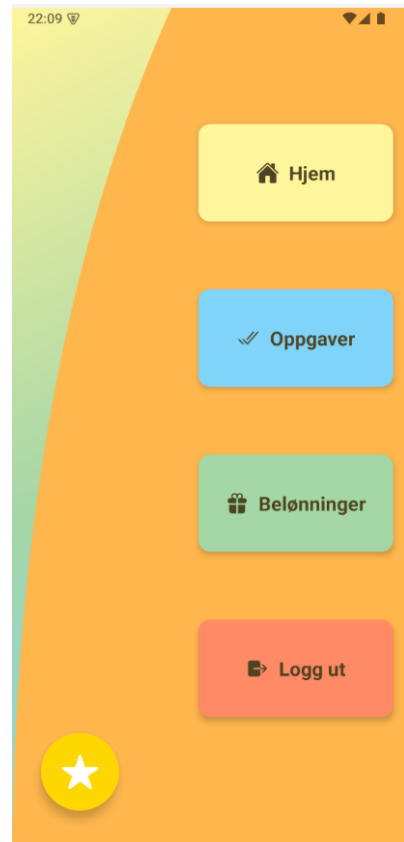


Figure 39: The menu popping up, when clicking the star button

10.4 App Access

Firebase Authentication is used to provide secure and user-friendly access to the app. A key requirement (AL1 and AL2) is that each parent user must verify their email and be linked to a "household" before accessing core functionality.

User Registration and Initial Setup

During registration, a Firebase user is created with email and password. A verification email is sent, and access is blocked until confirmed. Once registered, a Firestore document is created at `parents/{uid}` with profile info and `householdId = null`.

The household is created later, after email verification. The logic is encapsulated in `registerUser` in `authService.ts`.

```

1  const userCredential = await createUserWithEmailAndPassword(auth, email, password);
2  const uid = userCredential.user.uid;
3
4  const parent: Parent = {
5    id: uid,
6    firstName,
7    householdId, // null
8    avatar,
9  };
10
11 await setDoc(doc(db, 'parents', uid), parent);

```

Listing 4: Creating a Firebase Auth user and Firestore `Parent` document

After the first verified login, the user is guided through household setup, at which point the `householdId` is updated.

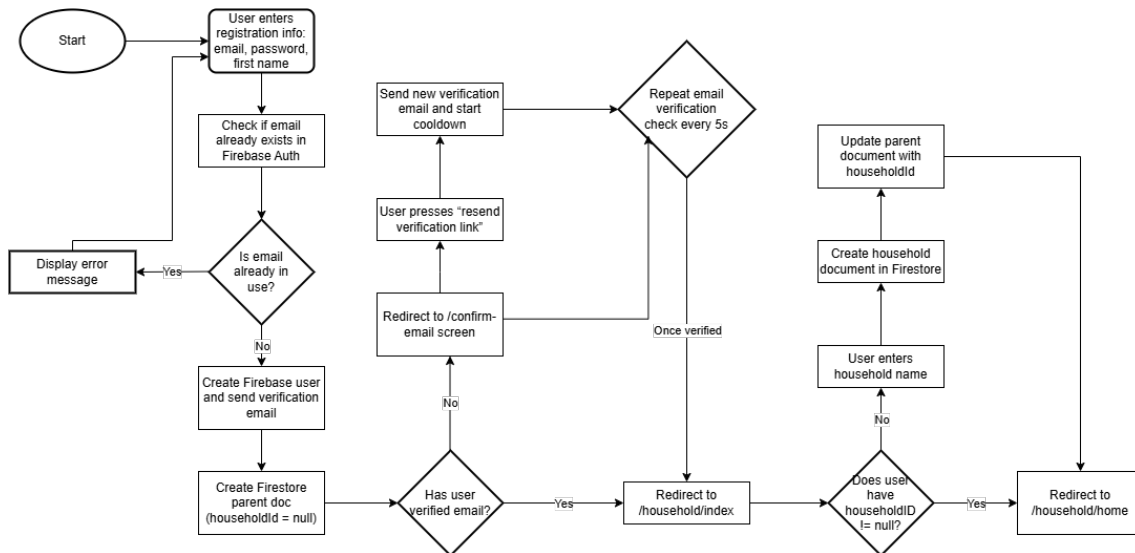


Figure 40: Registration and household creation flow in FortGjort

Email Confirmation and Routing Logic

Email verification is enforced through routing logic and a polling system. The custom hook `useEmailVerification()` checks every 5 seconds whether the email has been verified. Once verified, polling stops and the user is redirected to `/household`.

```
1 useEffect(() => {
2   const interval = setInterval(async () => {
3     const user = auth.currentUser;
4     if (user) {
5       await user.reload();
6       if (user.emailVerified) {
7         clearInterval(interval);
8         onVerified();
9       }
10    }
11  }, 5000);
12  return () => clearInterval(interval);
13 }, [auth.currentUser, onVerified]);
```

Listing 5: Polling logic in `useEmailVerification()`

A resend button is available with a 60-second cooldown.



Figure 41: Email confirmation screen with resend option

After login, the app checks if the email is verified. If not, the user is redirected to the confirmation screen. If verified, the app checks Firestore for `householdId` and routes accordingly:

- `null`: route to `/household`
- `valid`: route to `/household/home`

```

1  const handleLogin = async () => {
2    try {
3      const uid = await loginUser(email, password);
4      await auth.currentUser?.reload();
5
6      if (!auth.currentUser?.emailVerified) {
7        router.replace('/(auth)/confirm-email');
8        return;
9      }
10
11     const householdId = await getHouseholdIdForUser(uid);
12     router.replace(householdId ? '/(household)/home' : '/(household)');
13   } catch (e: any) {
14     Alert.alert('Login failed', e.message);
15   }
16 };

```

Listing 6: Login logic with routing based on verification and household membership

Pin-Registration

One of the security measures integrated into the app is the ability for parents to protect access to parent-specific views using a 4-digit PIN code. This prevents children from accessing restricted parental areas. During development, it was decided that this feature would be optional rather than enabled by default. Users can activate or deactivate PIN protection through the settings screen, where all PIN-related functionality is centralized. This approach was chosen to provide greater flexibility for parents and to simplify PIN management.

In line with the security analysis described in Section 9, the PIN is hashed using the SHA-256 algorithm via Expo Crypto before being stored in Firestore. The PIN is never stored or transmitted in plain text.

The resulting hash is saved as the `pinHash` attribute in the parent model and is set to `null` by default. Parents can enable PIN protection by setting a code via the settings interface. Once configured, the PIN must be entered each time the parent accesses restricted content.

When the user navigates to the PIN settings screen, the app checks whether a PIN has already been configured by querying the parent's document in Firestore. Depending on the presence of a `pinHash`, the interface dynamically displays the option to either create, update, or remove the PIN.

The snippet below illustrates this logic, including how the PIN is securely hashed and stored:

```

1 // Checks if parent has set a PIN
2 export const fetchParentHasPin = async (parentId: string): Promise<boolean> => {
3     const snap = await getDoc(doc(db, 'parents', parentId));
4     if (!snap.exists()) throw new Error('Parent not found');
5     return !!snap.data()?.pinHash;
6 };
7
8 // Hashes the PIN using SHA-256
9 export const hashPin = async (pin: string) =>
10     await Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256, pin);
11
12 // Stores the hashed PIN in Firestore
13 export const setPin = async (parentId: string, pin: string) => {
14     const hash = await hashPin(pin);
15     await updateDoc(doc(db, 'parents', parentId), { pinHash: hash });
16 };

```

Listing 7: Depending on the presence of a pinHash, the interface dynamically displays the option to either create, update, or remove the PIN. The PIN get securely hashed and stored

Each time the parent avatar is tapped in household home screen, the application performs a check to determine whether the parent has configured a PIN. If a pinHash exists in Firestore, indicating that a PIN has been set, a modal is shown prompting the user to enter the 4-digit code. If no PIN is present, the parent is navigated directly to the parent profile view.

This logic is shown in simplified form below:

```

1 const requiresPin = await fetchParentHasPin(authUid);
2
3 if (requiresPin) {
4     setShowPinModal(true); // + show modal
5 } else {
6     router.push('/:parent)/(tabs)/profile'); // + navigate directly
7 }

```

Listing 8: The application performs a check to determine whether the parent has configured a PIN.

A sequence diagram was also created to illustrate the flow of how a parent enters "parent mode." The diagram shows how the system first checks whether the parent has an ID. If not, the parent is prompted to enter an ID, which is then verified before the parent is granted access and redirected accordingly.

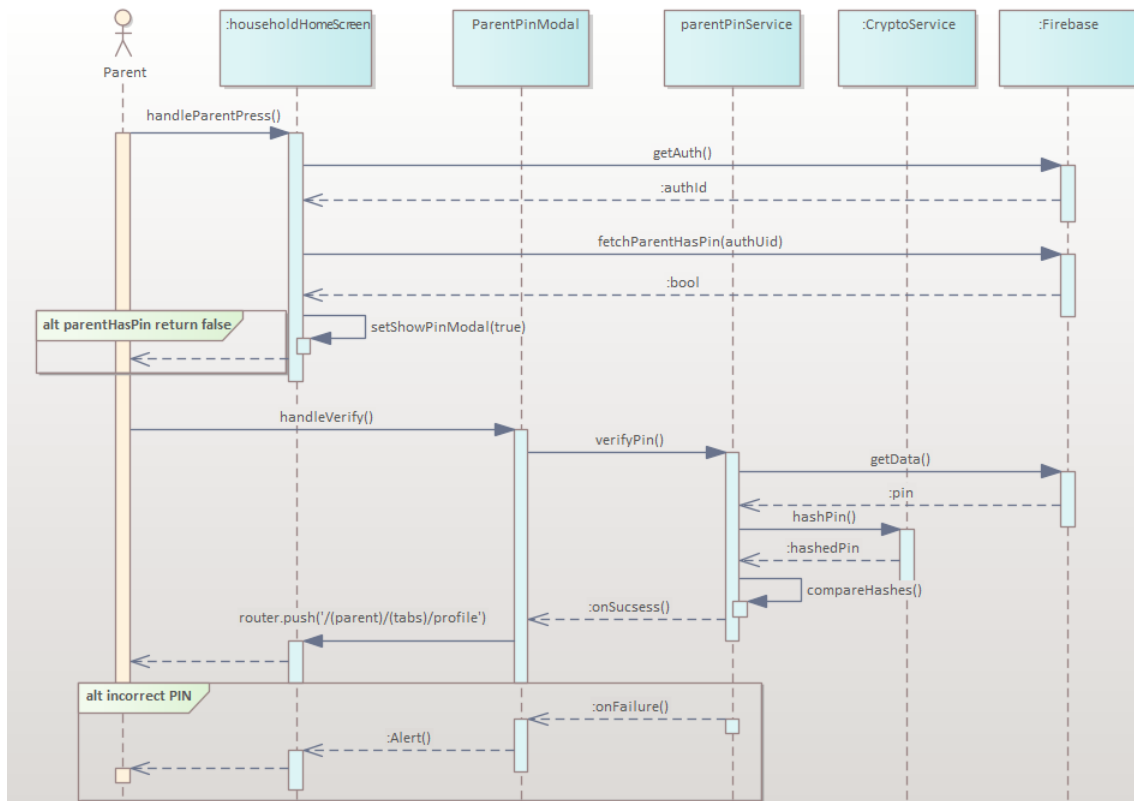


Figure 42: Sequence diagram illustrating the flow of a parent logging into the parent view.

In Figures 43, 44, and 45, the different PIN-related screens in the app are illustrated. These show how a parent can set a new PIN, update or remove an existing one, and how the PIN entry modal appears when attempting to access the parent profile.

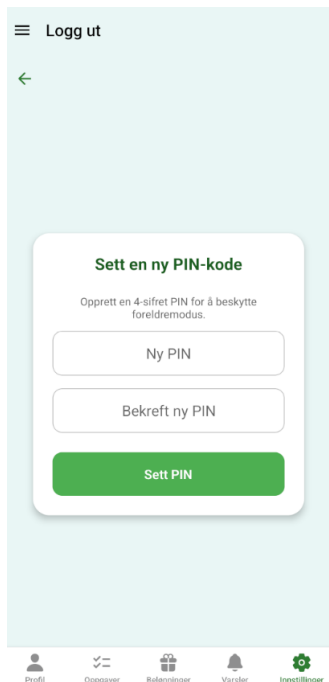


Figure 43: Set new PIN

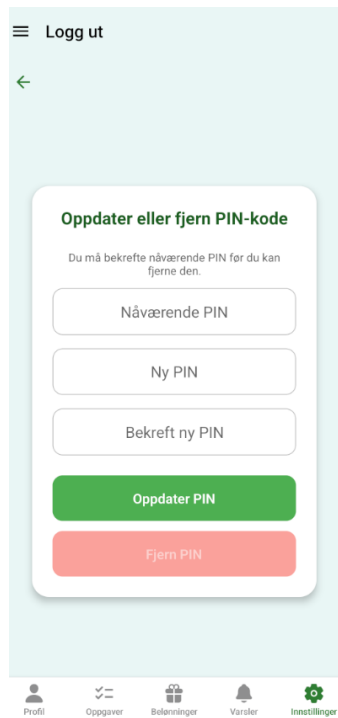


Figure 44: Screen for when parent has a PIN

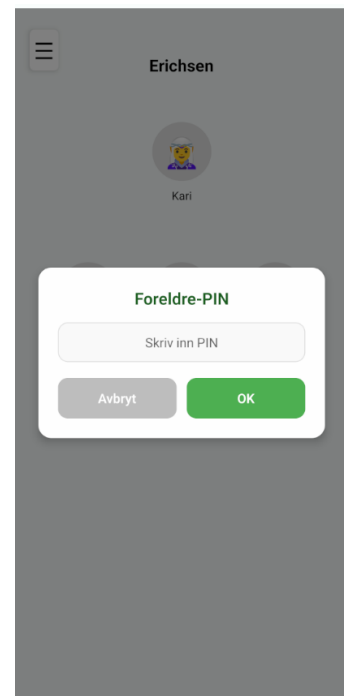


Figure 45: Enter PIN, to enter parent view

Session Persistence and Auto-Redirect

To enhance usability, the `WelcomeScreen` component uses Firebase's `onAuthStateChanged()` to check if a user is already logged in. If so, the user is redirected directly to the household screen, avoiding manual login.

This logic runs in a `useEffect` hook on component mount. If no user is found, a loading spinner is displayed until the check completes.

- Maintains session across restarts
- Skips unnecessary login steps
- Creates a smoother, user-aware experience

```
1 useEffect(() => {
2   const unsubscribe = onAuthStateChanged(auth, (user) => {
3     if (user) {
4       router.replace('/(household)');
5     } else {
6       setIsLoading(false);
7     }
8   });
9   return () => unsubscribe();
10 }, []);
```

Listing 9: Session detection in `WelcomeScreen.tsx`

While checking, a loading indicator is shown:

```
1 if (isLoading) {
2   return (
3     <View style={{ flex: 1, justifyContent: 'center', alignItems: 'center' }}>
4       <ActivityIndicator size="large" />
5     </View>
6   );
7 }
```

Listing 10: Loading state during authentication check

This logic ensures seamless continuation of the session and avoids unnecessary navigation steps.

10.5 Task Management

A central requirement in the application was to implement a flexible and intuitive task system where parents can create tasks, children can complete tasks assigned to them, and parents can validate the work. Each task awards points to the child once approved. There was also required support for recurring tasks, and safeguards against data loss from premature deletion.

Creating a Task

User stories TM2 and TM3 describe that a parent should be able to create a task and assign it to one or more children, so that the children can see the task in their version of the app. TM3 further specifies that it should be possible to create recurring tasks that appear on specific weekdays.

Before a task can be saved, the code checks that a name, a number of points, and at least one assigned child are provided. If any of these are missing, the operation is aborted. The parent can choose whether the task should be recurring, which

weekdays it should repeat on, or simply set a one-time deadline. When saved, one individual task is created for each selected child, and each task is linked to both the household and the specific child in the database.

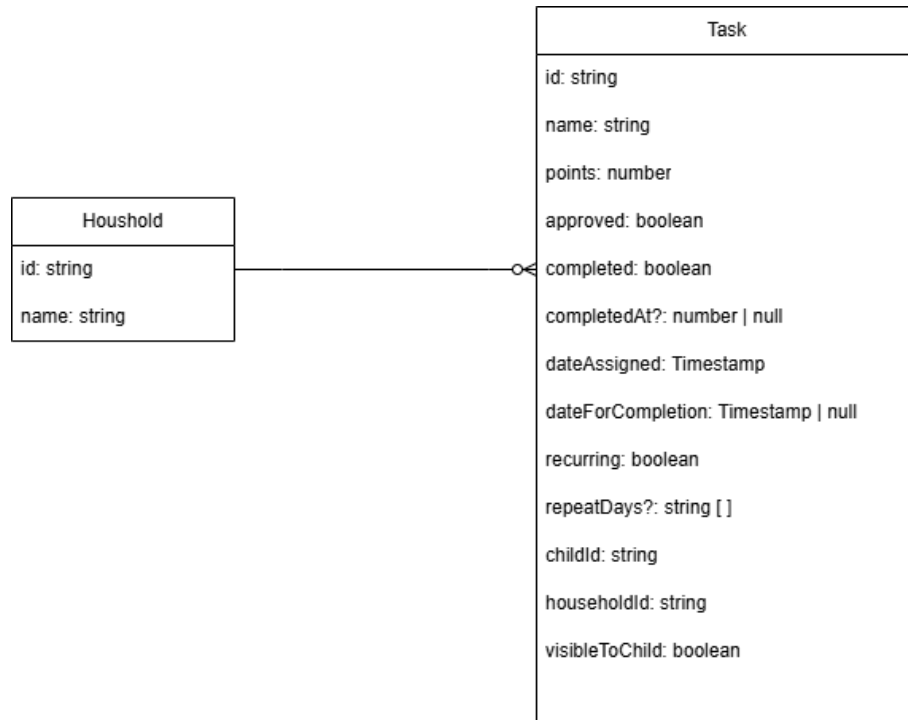


Figure 46: Relationship between tasks and household

Recurring Tasks

When creating a recurring task, the application calculates the dates on which the task should appear within the next 30 days, based on the selected weekdays. This is done using the `getRecurringDates` function, which filters all dates in the interval based on their weekday.

The code below shows how this is implemented, including a simplified example of task creation.

```

1  const weekdayMap = {
2    'SØ': 0, 'MA': 1, 'TI': 2, 'ON': 3, 'TO': 4, 'FR': 5, 'LØ': 6,
3  };
  
```

Listing 11: Mapping weekday codes to JavaScript's numeric day values

Function that returns dates within a range that match the selected weekdays:

```

1  const getRecurringDates = (repeatDays, start, end) =>
2      eachDayOfInterval({ start, end }).filter(date =>
3          repeatDays.includes(
4              Object.keys(weekdayMap).find(key => weekdayMap[key] === date.getDay())
5          )
6      );

```

Listing 12: Returns dates within a range that match the selected weekdays

When creating a task:

```

1  if (recurring && repeatDays.length > 0) {
2      const today = new Date();                // Start date: today
3      const end = new Date();                  // End date: 30 days from now
4      end.setDate(today.getDate() + 30);
5
6      // Find dates that match selected weekdays (e.g., 'MA', 'FR')
7      const recurringDates = getRecurringDates(repeatDays, today, end);
8
9      // Use the first matching date as the due date
10     if (recurringDates.length > 0) {
11         finalDateForCompletion = recurringDates[0];
12     }
13 }

```

Listing 13: Creating a task

Filtering tasks

Tasks follow a simple but flexible state model, centered around the fields `completed` and `approved`. Attributes such as `visibleToChild` and `dateForCompletion` were used to control task visibility and filtering logic. Tasks visible to the child must be marked as visible and not approved, and must be either incomplete but still within the deadline, or completed but not yet approved.

This is implemented in the `shouldShowInChildView` function:

```

export const shouldShowInChildView = (task: Task): boolean => {
  const now = new Date();
  const deadline = getDeadline(task.dateForCompletion);
  return (
    task.visibleToChild &&
    !task.approved &&
    (
      (task.completed && !task.approved) ||
      (!task.completed && deadline >= startOfDay(now))
    )
  );
};

```

Listing 14: Filtering tasks in child view

For parents, the filtering logic allows both completed and active tasks to be shown:

```

export const shouldShowInParentView = (task: Task): boolean => {
  const now = new Date();
  const deadline = getDeadline(task.dateForCompletion);
  if (task.completed && !task.approved) return true;
  if (!task.completed && deadline >= startOfDay(now)) return true;
  return !(task.completed && task.approved && deadline < startOfDay(now));
};

```

Listing 15: Filtering tasks in parent view

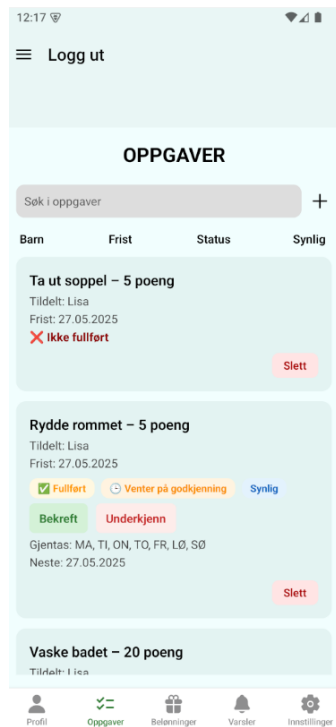


Figure 47: Listing tasks for parents

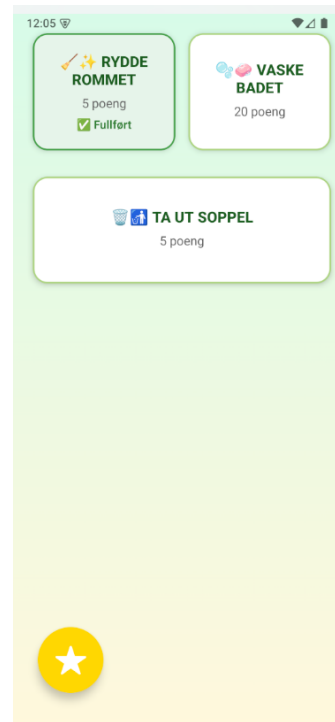


Figure 48: Listing tasks for children

Completion and Validation

When a child marks a task as completed, a confirmation modal is shown. Upon confirming, the task is updated in Firestore, and the completed attribute is set to true. After this, the parent can either approve or reject the task. If the parent rejects the task, the completed attribute is reset to false. If the parent approves the task, the approved attribute is set to true. In addition, the child is rewarded with the number of points associated with the task. These points are added to the total points the child has accumulated. The code snippet below shows an excerpt of what happens when a parent approves a task:


```

export const confirmApproval = async (householdId: string, task: Task) => {
  const taskRef = doc(db, collections.taskDocPath(householdId, task.id));
  await updateDoc(taskRef, { approved: true });

  const childRef = doc(db, collections.childDocPath(householdId, task.childId!));
  const childSnap = await getDoc(childRef);
  const currentPoints = childSnap.data()?.points ?? 0;

  await updateDoc(childRef, {
    points: currentPoints + task.points,
  });
};

```

Listing 16: An excerpt of what happens when a parent approves a task

Figure 49 shows a state diagram illustrating how a task transitions between different states.

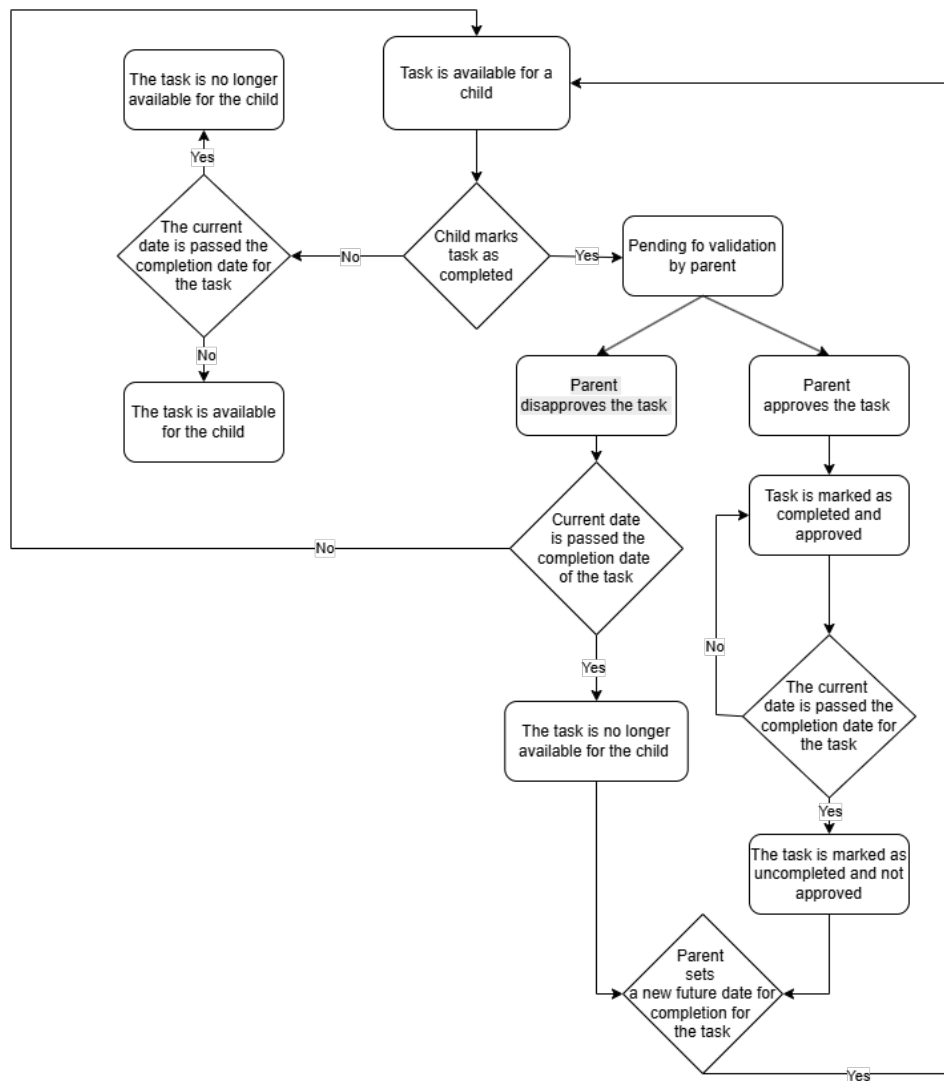


Figure 49: State diagram for a task

Deletion and Safety Measures

A key concern was to prevent task deletion if a child had marked it as completed but the parent had not yet reviewed it. To ensure this, there were added a safety check:

```

if (task.completed && !task.approved) {
  Alert.alert('Cannot delete', 'This task is completed but not yet approved.');
```

```

  return;
}

```

Listing 17: Checking if a task can get deleted

This prevents accidental deletion and ensures that the child's work is validated, and is shown in Figure 50.

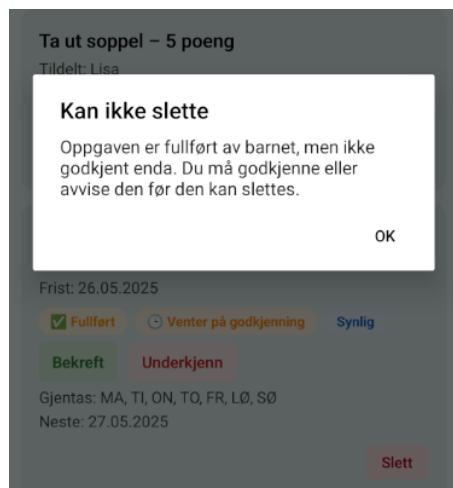


Figure 50: Trying to delete a completed, but pending task

10.6 Reward system

The reward system enables parents to create incentives and children to redeem them using earned points. This section describes user stories RS1–RS5, RS7, CFR1, and CFR2, covering reward creation, purchase, tracking, and usage toggling.

Creating a Reward

Parents can add rewards by opening a modal via the `AddButton`. On submission, a reward object is saved in the household's `rewards` subcollection in Firestore. This ensures rewards are namespaced per household:

```

const newReward: Omit<Reward, "rewardID"> = {
  name: rewardData.name,
  pointPrice: rewardData.pointPrice,
  addedBy: user.uid,
};
await addReward(householdId, newReward);

```

The following diagram shows how the reward is structured and stored in Firestore:

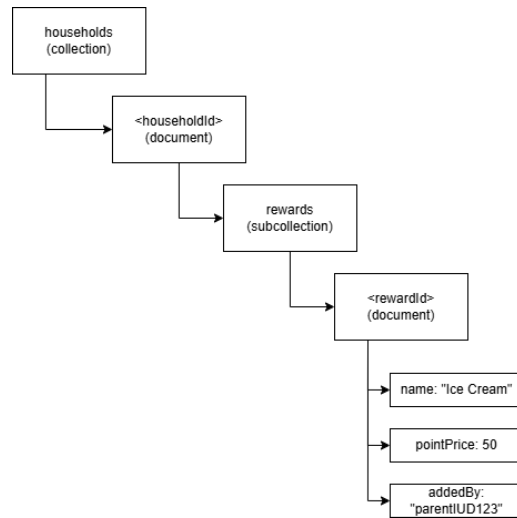


Figure 51: Firestore structure for rewards per household

The sequence diagram below illustrates the steps involved when a parent adds a new reward. A high resolution version of this diagram can be found in Appendix K or be directly accessed here: *Add Reward Sequence Diagram.svg*

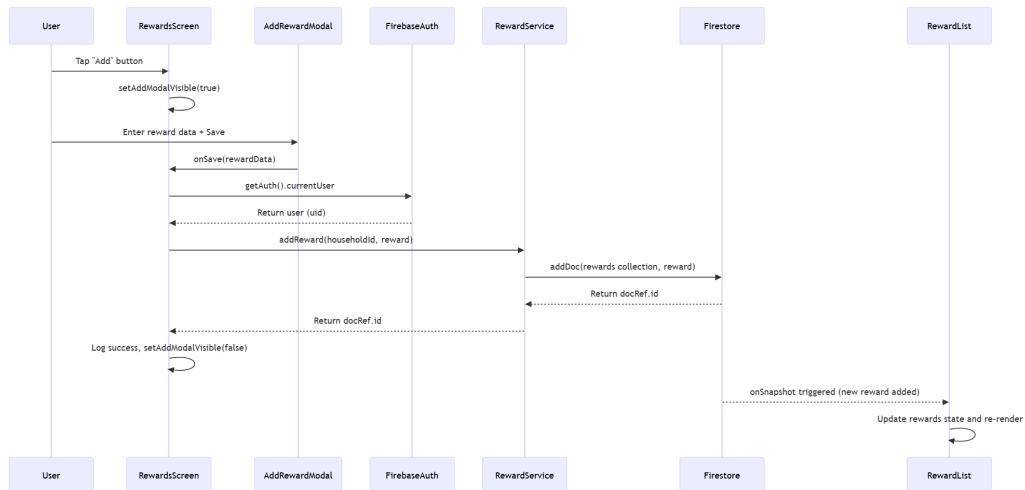


Figure 52: Sequence diagram showing the process of adding a reward

Realtime updates are handled via Firestore's `onSnapshot()`:

```
const unsubscribe = onSnapshot(rewardsRef, (snapshot) => {
  const fetchedRewards = snapshot.docs.map((doc) => ({
    rewardID: doc.id,
    ...doc.data(),
  }));
  setRewards(fetchedRewards);
});
```

This keeps the UI in sync without manual refreshes.

Purchasing a Reward

Children can exchange their earned points for rewards. This process updates both the Firestore database and the user interface in real time.

When a child selects a reward and confirms the purchase:

- The app checks if the child has enough points.
- If so, points are deducted from the child's Firestore record.
- A new document is added to `purchasedRewards` under the child.
- If not, a modal notifies the user of insufficient points.

This logic is implemented in `purchaseReward()` in `rewardService.ts`, and reflected instantly in the UI.

```
1  await updateDoc(childRef, {
2    points: currentPoints - reward.pointPrice,
3  });
4
5  await addDoc(
6    collection(db, `households/${householdId}/children/${childId}/purchasedRewards`),
7    {
8      rewardId: reward.rewardID,
9      name: reward.name,
10     pointPrice: reward.pointPrice,
11     used: false,
12   }
13 );
```

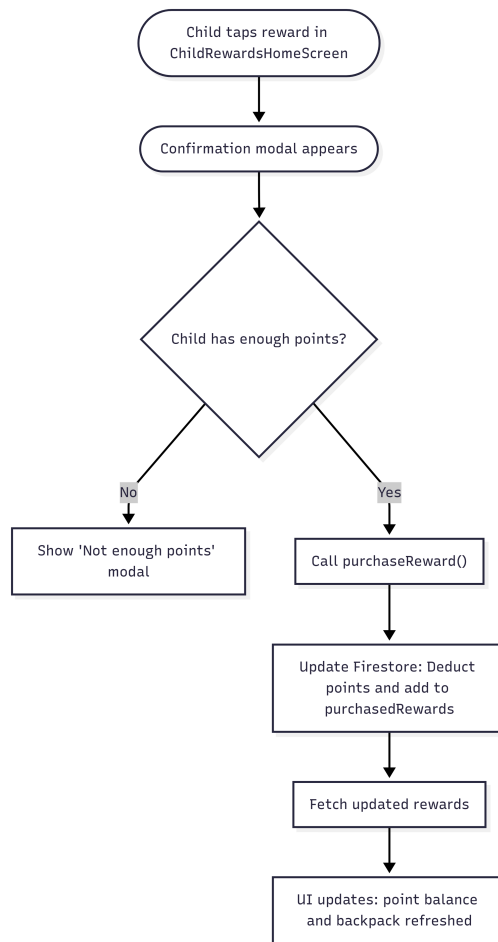


Figure 53: Flow chart of a child performing a reward purchase

Backpack View and Points Update

After a reward is purchased, it appears in the child’s backpack—a modal showing all rewards from the `purchasedRewards` subcollection where `used == false`.



Figure 54: UI showing purchased rewards ("backpack")

`BackpackModal.tsx` displays these rewards in a circular grid. If the list is empty, it shows the message "Sekken er tom".

```

1 {rewards.length === 0 ? (
2   <Text style={styles.emptyText}>Sekken er tom</Text>
3 ) : (
4   <View style={styles.rewardGrid}>
5     {rewards.map((reward) => (
6       <View key={reward.id} style={styles.rewardBubble}>
7         <Text style={styles.rewardEmoji}>emoji emoji</Text>
8         <Text style={styles.rewardText}>{reward.name}</Text>
9       </View>
10     ))}
11   </View>
12 )}
```

When a reward is purchased, points are immediately deducted and the new balance is shown via a "POENG" badge:

```

1 setPoints(prev => prev - selectedReward.pointPrice);
```

To stay in sync with Firestore, the backpack is updated by refetching all unused rewards:

```

1 const updatedPurchases = await fetchPurchasedRewards(householdId, childId);
2 const unused = updatedPurchases.filter((r) => !r.used);
3 setPurchasedRewards(unused);
```

This ensures the UI reflects the latest state after every purchase.

Marking Rewards as Used or Unused

Parents have the ability to mark a child's reward as either *used* or *unused*. This is achieved through the `RewardForChildScreen.tsx` interface, where a list of purchased rewards is presented. Tapping on an item opens the `UpdateRewardUsageModal`, which allows toggling the usage status via a switch component.

The status change is immediately persisted in Firestore using the following update logic:

```
1  const ref = doc(db,
2    `households/${householdId}/children/${childId}/purchasedRewards/${rewardId}`);
3  await updateDoc(ref, { used });
```

This update ensures that the field `used` is correctly set to either `true` or `false`, depending on the user's interaction.

To trigger this update, the following toggle logic is used inside the modal:

```
1  // UpdateRewardUsageModal.tsx
2  const toggleUsed = async () => {
3    const newValue = !isUsed;
4    setIsUsed(newValue);
5
6    try {
7      await updateRewardUsage(householdId, childId, rewardId, newValue);
8      onChange(); // Refresh local UI
9    } catch (error) {
10      console.error('Kunne ikke oppdatere reward:', error);
11    }
12  };
```

After the update, the modal calls `refreshRewards()` to fetch the latest state and reflect the change in the UI. The color-coded label "Brukt" or "Ikke brukt" visually distinguishes between completed and unredeemed rewards.

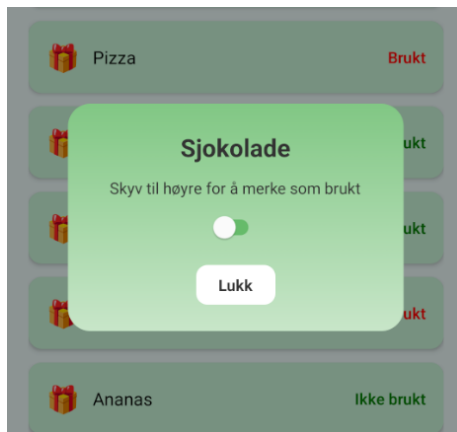


Figure 55: Toggle reward to used

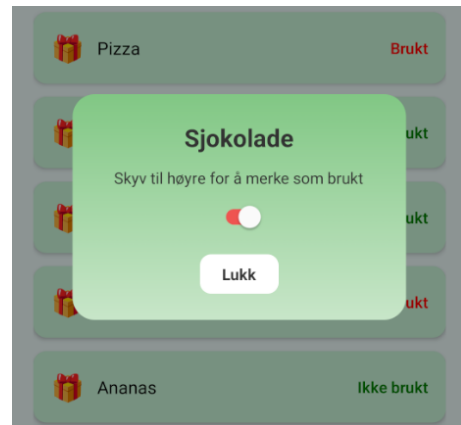


Figure 56: A reward has been toggled to used, and the status of the top reward, "Sjokolade", has updated in the UI behind the modal

11 Testing and validation

The automated functional tests were developed based on user stories. Each test was written using Maestro, a framework in which user interface interactions are simulated step by step. The tests are structured in YAML format, and each one executes a predefined sequence of actions such as tapping buttons, entering text, and validating the visibility or content of UI elements.

The purpose of these tests is to verify that the application's functionality behaves as expected. For each test case, specific UI elements are checked to ensure that they are visible or contain the correct data. If the expected condition is not met, the test is marked as "Failed" or "Partially Passed". If the condition is met, the test is marked as "Passed".

Figures 57 and 58 provide an overview of the test results. Comprehensive descriptions and step-by-step execution details for each test case are available in Appendix L, and can also be accessed directly via the following link: *Test Report*.

Maestro Testing				
Test Case ID	Use Case ID	Description	Pass/Fail/Not Executed	Test Date
TC01	AL1	As a parent, I want to create an account to access the app's features.	Passed	26.05.2025
TC02	AL2	As a parent, I want to create a household so my family can be connected under one account.	Passed	26.05.2025
TC03	AL3	As a parent, I want to log in with a username and password to access the app's features.	Passed	26.05.2025
TC04	AL7	As a parent, I want to log out of the app.	Passed	27.05.2025
TC05	AL8	As a parent, I want to log out of parent user.	Passed	27.05.2025
TC06	CA1	As a parent, I want to create one or more child accounts to my profile so that children can track tasks and rewards.	Passed	27.05.2025
TC07	CA2	As a parent, I want to see an overview of all children in the household to track their points and progress.	Passed	27.05.2025
TC08	CA3	As a parent, I want to manage/edit child accounts, including changing names or updating information.	Passed	27.05.2025
TC09	CA4	As a parent, I want to remove a child from the household if they no longer need access to the account.	Passed	27.05.2025
TC10	TM1	As a parent, I want to view all active tasks.	Passed	27.05.2025
TC11	TM2	As a parent, I want to create tasks and assign them to one or more children so they can view them in their profiles.	Passed	27.05.2025

Figure 57: Test cases 1-11

TC12	TM3	As a parent, I want to edit when a task should be completed, either one-time or recurring on selected weekdays.	Passed	27.05.2025
TC13	TM4	As a parent, I want to edit points on tasks so children know how much each task is worth.	Passed	27.05.2025
TC14	TM10	As a parent, I want to delete tasks that are no longer relevant.	Passed	27.05.2025
TC15	TM13	As a parent, I want to approve completed tasks so children receive their rewards.	Passed	27.05.2025
TC16	RS1	As a parent, I want to view all available rewards so I know what options are available for children.	Passed	27.05.2025
TC17	RS2	As a parent, I want to add new reward that children can "buy" with their earned points.	Passed	27.05.2025
TC18	RS3	As a parent, I want to edit the point cost for each reward.	Passed	27.05.2025
TC19	RS4	As a parent, I want to see which rewards my children have purchased.	Passed	27.05.2025
TC20	RS5	As a parent, I want to mark a reward as "used" once the child has received it.	Passed	27.05.2025
TC21	RS7	As a parent, I want to delete rewards that are no longer relevant.	Passed	27.05.2025
TC22	CFP1	As a child, I want to view my profile with my name, picture, and points so I can keep track of my progress.	Passed	27.05.2025
TC23	CFP4	As a child, I want to see how many points I've earned and how many I have available for rewards, so that I know how many points I am able to use in store.	Passed	27.05.2025
TC24	CFT1	As a child, I want to mark tasks as complete, so that a parent can approve if they are done or not.	Passed	27.05.2025
TC25	CFR1	As a child, I want to view a list of available rewards so that I can purchase with my points, so that I can decide whether to buy something immediately or save my points for a future purchase.	Passed	27.05.2025
TC26	CFR2	As a child, I want to see which rewards I have already earned, so that I can keep track of already earned rewards.	Passed	27.05.2025
TC27	PS01	As a parent, I want to edit my profile information such as avatar and name.	Passed	27.05.2025
TC28	PS02	As a parent, I want to edit my user password.	Passed	27.05.2025

Figure 58: Test cases 12-28

12 Discussion

12.1 Process

After deciding on the domain—a household chore app for families, the focus shifted to laying a solid foundation for development. Time was spent designing the database, defining functional requirements, creating Figma views, and writing use case descriptions. This structured approach provided clarity and direction when implementation began.

Looking back, the upfront planning phase was both a strength and a challenge. It helped align the team and provided a solid foundation for development. However, as the app evolved, new insights required frequent updates to Figma views and user stories, updates that were time-consuming, but mostly maintained. In contrast, the use case descriptions were deprioritized and rarely updated, meaning many no longer reflect the final version of the app.

During development, the team found a steady rhythm through regular meetings and short sprints. However, implementing key features, such as user login, parent profiles, and the household setup—proved a bottleneck, as other components depended on them. With more detailed planning, this phase might have run more smoothly. Still, waiting periods were used effectively for tasks like updating documentation.

12.2 Product

Choice of Technologies

At the start of development, relevant technologies were reviewed based on the project’s goals. While none were fully familiar, the chosen tools had been introduced through course lectures and exercises. Despite a reasonably thorough investigation, as outlined in Section 5, limited time and experience led to a focus on technologies with some prior exposure. In hindsight, other technologies might have been better suited for certain parts of the solution, but time constraints made implementation a higher priority than continued exploration.

Reflections on Database Structure

The current Firestore structure was designed to support a functional MVP, but several architectural choices were reconsidered during development. Notably, children, tasks, and rewards are nested under the household document, while the parent is stored separately and linked via a foreign key. This decision stemmed from the creation order since the parent is created before the household, nesting it was technically inconvenient. While the trade-off made sense at the time, alternatives such as restructuring the onboarding flow or using a temporary placeholder household could have been considered.

Another key discussion point was the placement of ‘tasks’. In the current model, tasks are grouped under the household and linked to children via ‘childId’. This works functionally, but requires explicit filtering and adds complexity to child-specific queries. Structuring tasks directly under each child could have made the data model more intuitive and better aligned with access patterns, especially since tasks are inherently tied to the child who performs them. That said, storing all tasks centrally under the household has practical benefits, particularly for parents, who often need an overview of all tasks across children.

12.3 Implementation

Parent profile view

As outlined in Section 5.6, key UI and UX principles for both adults and children were identified during the research phase and applied where relevant during development. The parent interface uses a clean, minimalistic design with a bottom tab bar that offers familiar and predictable navigation. Each screen focuses on a single function like tasks, rewards, or settings—to reduce cognitive load. Color-coded status indicators in the task view support quick interpretation. However, some screens, such as ‘Rewards’ and ‘Settings’, lack visual hierarchy. Uniform button styles may cause confusion, and clearer visual cues—like contrasting colors or varied typography—could help highlight key actions. These improvements could have been implemented with more time available.

Child profile view

The child interface prioritizes simplicity and clarity and uses a colourful design. A floating menu with large, colourful buttons and clear icons replaces traditional tab navigation, supporting intuitive use and reducing cognitive load. To prevent confusion, submenus are avoided, giving direct access to core features—Home, Tasks, Rewards, and Logout. Motivational elements like earning points and visual feedback (e.g., confetti) help increase engagement, in line with design recommendations for children. The interface also eliminates the need for typing, relying entirely on touch-based interaction.

That said, features such as a virtual helper or sound-based feedback—often recommended for improving engagement and guidance were not implemented due to time constraints. These remain potential areas for future enhancement, especially considering their proven effectiveness in similar applications aimed at children.

Code structure

Early in the project, research was done to define a clear folder structure and naming conventions to support maintainable and well-documented code. However, as time pressure increased, consistency in code architecture suffered. Varying implementa-

tion approaches and lapses in team communication led to structural and stylistic inconsistencies. In hindsight, maintaining architectural discipline should have been given higher priority.

That said, the project maintained a relatively clear file structure overall. Styles were consistently separated, and most code followed a logical folder hierarchy. Database-related logic was also placed in service files within their respective folders. While the foundation is solid, internal consistency within some components could be improved. A more uniform approach to UI structure and architectural patterns would have further enhanced code quality and maintainability.

12.4 Challenges

Automatic End-to-End Testing

The team originally intended to use automatic end-to-end (E2E) testing to simplify validation after implementing changes, and to avoid manually testing each use case. Initial attempts were made with Jest and Detox, but configuring Detox proved difficult and time-consuming. Eventually, the team switched to Maestro, which allowed writing tests in YAML—a format that felt more intuitive than JavaScript for describing user flows.

While Maestro made it easier to get started with E2E testing, several challenges quickly emerged:

- Tests were not isolated: running one test would alter the app’s state in a way that affected the next, leading to unpredictable results.
- Some UI interactions—such as scrolling until an element becomes visible—were hard to express in YAML.
- Test execution time was long, and when a single test failed, it was unclear whether the full suite should be re-run.

As a workaround, the team chose to run one test at a time and manually reset or update the Firestore database between runs to prepare for the next test. While this made test execution possible, it was far more time-consuming than expected.

One core challenge was that tests were written early, based on static Figma designs. As the UI evolved, tests had to be repeatedly rewritten, making them reactive rather than guiding development. This raised questions about the effectiveness of the approach.

Although the test suite was neither fully functional nor reusable in the end, the process provided valuable insight into test architecture, tool limitations, and the need for flexible testing strategies. In hindsight, focusing more on core features

rather than fragile UI tests may have been a better use of time in a fast-evolving codebase.

12.5 For the future

Shared household access

The original plan included support for shared household access, where two parents could manage the same household. This could be done through shared login or by linking separate parent accounts. As development progressed, this feature was postponed due to time constraints and complexity.

The main challenge was designing a secure and clean way to manage user roles and permissions. Implementing this would have required backend changes that were outside the project's scope. With more time, viable solutions could likely have been found. For now, this remains a priority for future development.

Camera function for task verification

A planned feature was letting children take photos to verify completed tasks. This was excluded due to the complexity of handling camera permissions, secure media storage, and backend scalability. While technically feasible, it required more time and research than was available. Still, it remains a valuable addition for future versions.

Notification

A notifications tab was added to the parent view as part of the app's navigation, but the functionality was not fully implemented due to time constraints and project scope. As it wasn't essential for the MVP, the feature was postponed. The tab remains to simplify future implementation, with the goal of notifying parents when a child completes a task, redeems a reward, or similar—laying the groundwork for improved in-app communication.

Alternative Login Methods

The registration view includes a Facebook login button, which is currently non-functional. This feature was intended to be implemented, and research showed that Firebase offers built-in support for Facebook authentication. However, due to time constraints, integration was de-prioritized. Given that the functionality is well-supported by Firebase, it was considered something that could be added at a later stage without major effort.

13 Conclusion

The project aimed to simplify and motivate household task management for families with children. Although similar apps exist, early research showed that many are overly complex, require children to have email accounts, or lock key features behind paywalls. This led to the project’s vision: a free, user-friendly app that gives parents control while engaging children on their own terms.

A cross-platform mobile app was developed using React Native, Expo, and Firebase as the primary technologies. The team encountered several challenges due to limited prior experience with these frameworks, particularly React Native, which required a steep learning curve and impacted both development speed and feature prioritization. Despite this, a functional MVP was delivered, including core features like user management, task tracking, and a reward system.

A central focus throughout development was adapting the interface to suit different user groups. Parents access administrative features via a secure PIN code, while children interact with a simplified, colourful interface designed for their cognitive level and needs.

Collaboration was key to solving technical challenges and making design decisions. The project mainly followed a waterfall approach, with most planning done early, but incorporated some agile-inspired iteration and flexibility. Automated end-to-end testing proved unreliable, so testing was primarily manual—offering valuable insight into practical test strategies in a dynamic environment.

The application is now functional and ready for internal testing via Google Play. While not yet prepared for public release, this version allows for evaluation and further refinement. Planned improvements include support for notifications, shared parental access, image upload as proof of completed tasks, and a clear privacy policy with user consent features in line with GDPR.

Overall, the project resulted in a secure and user-centred application that supports collaboration between parents and children in an engaging and structured way. The process was marked by effective teamwork, continuous learning, and practical problem-solving. The final product demonstrates clear value for its intended users and offers a solid base for future refinement and feature expansion.

References

- [1] *Accessibility*. [Online; accessed 24. Feb. 2025]. Jan. 2025. URL: <https://developer.android.com/design/ui/mobile/guides/foundations/accessibility>.
- [2] Aphelia. *Best Backend for React Native*. 2023. URL: <https://www.aphelia.co/blogs/best-backend-for-react-native>.
- [3] Appfabrikken AS. *samboer.app*. Version 1.6.6. 2021. URL: <https://appfabrikken.no/samboer-appen>.
- [4] *Build an offline-first app*. [Online; accessed 23. Feb. 2025]. Feb. 2025. URL: <https://developer.android.com/topic/architecture/data-layer/offline-first>.
- [5] Abdul Mateen Chughtai. “Understanding Offline-First Architecture in Android”. In: *Medium* (July 2023). URL: <https://medium.com/@abdulmateenchughtai/understanding-offline-first-architecture-in-android-801cad2e81e8>.
- [6] *Component-Based Architecture*. [Online; accessed 4. Apr. 2025]. Apr. 2025. URL: https://www.tutorialspoint.com/software_architecture_design/component_based_architecture.htm.
- [7] *Cordova Performance Tips*. URL: <https://cordova.apache.org/docs/en/11.x/guide/next/>.
- [8] CreatorsJourney AS. *Ukelønn*. Version 2.0.3. 2024. URL: <https://https://www.ukelonn.no/>.
- [9] *Crypto*. [Online; accessed 27. May 2025]. May 2025. URL: <https://docs.expo.dev/versions/latest/sdk/crypto>.
- [10] *Django*. [Online; accessed 16. Feb. 2025]. Feb. 2025. URL: <https://www.djangoproject.com>.
- [11] Sanket R. Durgekar et al. “A Review Paper on Design and Experience of Mobile Applications”. In: *EAI Endorsed Transactions on Scalable Information Systems* (Jan. 2024). ISSN: 2032-9407. DOI: 10.4108/eetsis.4959.
- [12] Editor. “The Good and the Bad of Express.js Web Framework”. In: *AltexSoft* (Nov. 2024). URL: <https://www.altexsoft.com/blog/expressjs-pros-and-cons>.
- [13] Omar Elgabry. “Component Based Architecture - OmarElgabry’s Blog - Medium”. In: *Medium* (Dec. 2021). ISSN: 3323-7348. URL: <https://medium.com/omarelgabrys-blog/component-based-architecture-3c3c23c7e348>.
- [14] *Enabling Offline Capabilities | Firebase Realtime Database*. [Online; accessed 23. Feb. 2025]. July 2022. URL: <https://firebase.google.com/docs/database/flutter/offline-capabilities>.
- [15] *Expo Router notation*. [Online; accessed 26. May 2025]. May 2025. URL: <https://docs.expo.dev/router/basics/notation>.
- [16] *Express - Node.js web application framework*. [Online; accessed 23. Feb. 2025]. Feb. 2025. URL: <https://expressjs.com>.
- [17] *Families and households*. [Online; accessed 4. Mar. 2025]. Sept. 2024. URL: <https://www.ssb.no/en/befolkning/barn-familier-og-husholdninger/statistikk/familier-og-husholdninger>.

- [18] Jinal Gajera. *A Comprehensive Guide to Backend Options for React Native Development*. 2022. URL: <https://medium.com/@jinalg.tagline/a-comprehensive-guide-to-backend-options-for-react-native-development-8d3fb1e2da5c>.
- [19] Jonathan Gamble. “Supabase vs. Firebase: The Dominant Database Platform”. In: *Telerik Blogs* (Mar. 2024). URL: <https://www.telerik.com/blogs/supabase-vs-firebase-dominant-database-platform>.
- [20] GeeksforGeeks. “7 Best Frontend Framework for Django”. In: *GeeksforGeeks* (Dec. 2024). URL: <https://www.geeksforgeeks.org/best-frontend-framework-for-django>.
- [21] GeeksforGeeks. “Top 7 Databases for Mobile App Development in 2025”. In: *GeeksforGeeks* (Jan. 2025). URL: <https://www.geeksforgeeks.org/databases-for-mobile-app-development/#2-sqlite>.
- [22] *Getting started*. [Online; accessed 22. May. 2025]. Apr. 2025. URL: <https://reactnative.dev/docs/getting-started>.
- [23] Gimi AB. *Gimi*. Version 10.2.0. 2023. URL: <https://gimi.com/no/>.
- [24] Google. *Flutter Documentation*. URL: <https://docs.flutter.dev/get-started/learn-flutter>.
- [25] *Household structure in the EU*. [Online; accessed 4. Mar. 2025]. Mar. 2025. URL: <https://ec.europa.eu/eurostat/web/products-statistical-working-papers/-/ks-ra-10-024>.
- [26] *Introduction to Expo Router*. [Online; accessed 26. May 2025]. May 2025. URL: <https://docs.expo.dev/router/introduction>.
- [27] *Ionic Utility Animations*. URL: <https://ionicframework.com/docs/utilities/animations>.
- [28] Mahil Jasani. “An Introduction to Realm Database - excellentwebworld - Medium”. In: *Medium* (Dec. 2021). URL: <https://medium.com/excellentweb/an-introduction-to-realm-database-2881f0f8c231>.
- [29] Elif İrem Kara Kadyrov. “Understanding the Essence of Component-Based Architecture”. In: *Medium* (June 2023). URL: <https://mobileappcircular.com/understanding-the-essence-of-component-based-architecture-5c6d4452f100>.
- [30] Radoslava Stankova Kraveva. “Designing an Interface For a Mobile Application Based on Children’s Opinion”. In: *International Journal of Interactive Mobile Technologies (iJIM)* 11.1 (Jan. 2017), pp. 53–70. ISSN: 1865-7923. DOI: 10.3991/ijim.v11i1.6099.
- [31] Kornel Kwiatkowski. *What is Expo?* 2024. URL: <https://pagepro.co/blog/what-is-expo-js/>.
- [32] Mirza Leka. “Express.js Basics & Beyond - Mirza Leka - Medium”. In: *Medium* (Jan. 2025). ISSN: 5960-1806. URL: <https://mirzaleka.medium.com/express-js-basics-beyond-59ed601e806b>.
- [33] Álvaro Manjarres Merino. *Firebase: An all-in-one platform for Web and Mobile application development*. 2024. URL: <https://www.sngular.com/insights/313/firebase>.
- [34] Microsoft. *Introduction to .NET MAUI*. 2024. URL: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/introduction>.
- [35] Microsoft. *What is .NET MAUI*. URL: <https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui>.

- [36] Microsoft. *Xamarin*. URL: <https://dotnet.microsoft.com/en-us/apps/xamarin>.
- [37] *Native Platform*. [Online; accessed 22. May. 2025]. Apr. 2025. URL: <https://reactnative.dev/docs/native-platform>.
- [38] *Navigation layouts in Expo Router*. [Online; accessed 26. May 2025]. Apr. 2025. URL: <https://docs.expo.dev/router/basics/layout>.
- [39] NEatkid kids chore tracker and reward chart, mchj. *neatkid*. Version 2.15.0. 2024. URL: <https://neatkid.io/>.
- [40] Hiten Pandya. "What Is Firebase?" In: *CSharp.com* (Feb. 2018). URL: <https://www.csharp.com/blogs/what-is-firebase>.
- [41] *Platform-Specific Code*. [Online; accessed 22. May. 2025]. Apr. 2025. URL: <https://reactnative.dev/docs/platform-specific-code>.
- [42] Amanda J. Pooley and Emily M. Thorne. *Children and chores: Is it worth the battle?* [Online; accessed 4. Mar. 2025]. Mar. 2025. URL: <https://www.rica.nsw.edu.au/resources/children-and-chores-it-worth-battle>.
- [43] Jakub Protasiewicz. "Top 10 Django Apps Examples". In: *Netguru* (Dec. 2024). URL: <https://www.netguru.com/blog/django-apps-examples>.
- [44] *React Native - Animations*. 2024. URL: <https://reactnative.dev/docs/animations>.
- [45] *Realm vs SQLite: The Best Database for Android App Building*. [Online; accessed 23. Feb. 2025]. Feb. 2025. URL: <https://www.cleveroad.com/blog/realm-vs-sqlite-what-is-the-best-database-for-android-app-development>.
- [46] Harisudhan. S. "Supabase vs Firebase: Detailed Comparison for Your Next Project". In: *Medium* (Nov. 2024). URL: <https://medium.com/@speaktoharisudhan/supabase-vs-firebase-detailed-comparison-for-your-next-project-ffcaaea30037>.
- [47] Aryan Singh. "Introduction to Django for backend development - Aryan Singh - Medium". In: *Medium* (Mar. 2023). ISSN: 2677-6862. URL: <https://medium.com/@codingwitharyan/introduction-to-django-for-backend-development-2db67768ee62>.
- [48] Clockwise Software. *Cordova vs React Native - A Comparison*. 2023. URL: <https://clockwise.software/blog/cordova-vs-react-native-comparison/>.
- [49] iRoid Solutions. "Mobile App Backend Development: Complete Guide - iRoid Solutions - Medium". In: *Medium* (Dec. 2024). URL: <https://medium.com/@blog.iroidsolutions/mobile-app-backend-development-complete-guide-3ce3a3a4aafe>.
- [50] Meta Open Source. *React Native Animated API*. 2023. URL: <https://reactnative.dev/docs/animated>.
- [51] Marcos Souza. *Flutter vs React Native's New Architecture Performance Benchmark*. 2023. URL: <https://marcosouz4.medium.com/flutter-vs-react-natives-new-architecture-performance-benchmark-c7c90ac8273e>.
- [52] Doug Stevenson. "What is Firebase? The complete story, abridged. - Firebase Developers - Medium". In: *Medium* (May 2020). URL: <https://medium.com/firebase-developers/what-is-firebase-the-complete-story-abridged-bcc730c5f2c0>.
- [53] *Supabase | The Open Source Firebase Alternative*. [Online; accessed 23. Feb. 2025]. Feb. 2025. URL: <https://supabase.com>.

- [54] Deanna L. Tepper, Tiffani J. Howell, and Pauleen C. Bennett. “Executive functions and household chores: Does engagement in chores predict children’s cognition?” In: *Aust. Occup. Ther. J.* 69.5 (Oct. 2022), pp. 585–598. ISSN: 1440-1630. DOI: 10.1111/1440-1630.12822. eprint: 35640882.
- [55] *UX Design for Children: How to Create a Product Children Will Love*. [Online; accessed 1. Apr. 2025]. Apr. 2025. URL: <https://www.eleken.co/blog-posts/ux-design-for-children-how-to-create-a-product-children-will-love>.
- [56] *UX Design for Kids: Principles and Recommendations | Ramotion Agency*. [Online; accessed 24. Feb. 2025]. Feb. 2024. URL: <https://www.ramotion.com/blog/ux-design-for-kids/#section-child-centric-ux-design>.
- [57] UXCam. *.NET MAUI vs React Native Performance Comparison*. 2023. URL: <https://uxcam.com/blog/net-maui-vs-react-native/>.
- [58] Uxpin. “9 Principles of Mobile App Design”. In: *Studio by UXPin* (Aug. 2023). URL: <https://www.uxpin.com/studio/blog/principles-mobile-app-design>.
- [59] Jennifer Breheny Wallace. “Why Children Need Chores”. In: *WSJ* (Mar. 2015). URL: <https://www.wsj.com/articles/why-children-need-chores-1426262655>.
- [60] Jennifer Breheny Wallace. “Why Children Need Chores”. In: *WSJ* (Mar. 2015). URL: <https://www.wsj.com/articles/why-children-need-chores-1426262655>.
- [61] *Web Animations API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Animations_API.
- [62] Whirlpool Corporation. “Want Kids To Thrive As Adults? Give Them Chores Today, Whirlpool Research Reveals”. In: *Cision PR Newswire* (July 2020). URL: <https://www.prnewswire.com/news-releases/want-kids-to-thrive-as-adults-give-them-chores-today-whirlpool-research-reveals-301098390.html>.
- [63] Elizabeth M. White, Mark D. DeBoer, and Rebecca J. Scharf. “Associations Between Household Chores and Childhood Self-Competency”. In: *J. Dev. Behav. Pediatr.* 40.3 (Apr. 2019), p. 176. ISSN: 0196-206X. DOI: 10.1097/DBP.0000000000000637.
- [64] *Who we are - Eurostat*. [Online; accessed 4. Mar. 2025]. Mar. 2025. URL: <https://ec.europa.eu/eurostat/about-us/who-we-are>.
- [65] Jeroen Willemsen et al. *OWASP Mobile Application Security Verification Standard (MASVS) v2.1.0*. <https://mas.owasp.org/MASVS>. Version 2.1.0. Released by OWASP Foundation on January 18, 2024. Licensed under Creative Commons Attribution-ShareAlike 4.0 International. 2024. URL: <https://mas.owasp.org/MASVS>.

A Product Vision

The complete Product Vision is available in the appendices folder here: *product-vision.pdf*

B Group Contract

The Group Contract is available in the appendices folder here: *group-contract.pdf*

C Product Info

Product info can be found in the appendices, here:

Username and Password for users: *product-information.pdf*

D Meeting Minutes

The complete set of Meeting Minutes is available in the appendices folder here: *meeting-minutes.pdf*

E Demo Video

The Demo Video showcasing the product is available in the appendices folder here: *demo-video.mp4*

F Git Shortlog

The Git Shortlog is available in the appendices folder here:

git.log

G Scrum List

The complete Scrum list can be found in the appendices folder here: *scrum-list.pdf*

H Requirements

The complete Requirements is available in the appendices folder here: *requirements.pdf*

I Figma Views

The complete Figma Views is available in the appendices folder here: *figma-views.pdf*

J Security Analysis

The complete Security Analysis is available in the appendices folder here: *security.pdf*

K Add Reward Sequence Diagram

A high resolution version of the Add Reward Sequence Diagram is available in the appendices folder here: *add-reward-sequence-diagram.svg*

L Test Report

The complete Test Report is available in the appendices folder here: *testing-and-validation.pdf*

M Entity Model

The final entity model is available in the appendices folder here: *entity-model.svg*

N Individual Reports

The individual reports can be found for each person in the appendices folder here:

Nancy Rønqist Erichsen: */individual_reports/nancy_report.pdf*

Adam Hazel: */individual_reports/adam_report.pdf*

Gunn Marita Jomås-Britten: */individual_reports/gm_report.pdf*

Nora Lior: */individual_reports/nora_report.pdf*

O Time Sheets

The time sheets can be found for each person in the appendices folder here:

Nancy Rønqist Erichsen: */time_sheets/time-sheet-nancy-erichsen.pdf*

Adam Hazel: */time_sheets/time-sheet-adam-hazel.pdf*

Gunn Marita Jomås-Britten: */time_sheets/time-sheet-gunn-marita.pdf*

Nora Lior: */time_sheets/time-sheet-nora-lior.pdf*

P Project Brainstorming

A visual representation of the initial project brainstorming can be found in the appendices folder here: *project_brainstorming.jpg*

Q Use Case Descriptions

A complete list of all Use Case Descriptions can be found in the appendices folder here: *usecase-descriptions.pdf*